

**A MULTI SENSOR INDOOR/OUTDOOR REAL TIME IOT HYBRID MODEL
APPROACH FOR FIRE DETECTION**

by

JOSE A. BURNIAS, B.S

THESIS

Presented to the Graduate Faculty of
The University of Texas at San Antonio
in Partial Fulfillment
of the Requirements
for the Degree of

MASTER OF SCIENCE IN COMPUTER ENGINEERING

COMMITTEE MEMBERS:

Jeff Prevost, Ph.D., Chair
Ram Krishnan, Ph.D.
Palden Lama, Ph.D.

THE UNIVERSITY OF TEXAS AT SAN ANTONIO
College of Engineering and Integrated Design
Department of Electrical and Computer Engineering
December 2024

DEDICATION

I would like to dedicate this thesis to my mother Enedelia De la Paz De Burnias, my father Jose Alberto Burnias Rios, and my brother Juan Manuel Burnias. They have always believed in me throughout all the challenges I have encountered in this phase of my academic journey and provided strength and support to keep moving forward.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude and respect to my advisor and mentor, Professor Jeff Prevost, Ph.D., for the unconditional support, patience, and invaluable knowledge throughout this pivotal phase of my academic journey. His guidance encouraged me to look at my studies from different perspectives that have helped me grow both personally and professionally, which is something I look forward to take with me into all my future endeavors.

I would also like to acknowledge my committee members, Professor Palden Lama, Ph.D. and Professor Ram Krishnan, Ph.D. for their invaluable support and guidance as part of my committee. They both made a significant impact on my graduate level studies and it was a privilege for them to add insightful comments and suggestions towards my work. Their profound expertise and contributions to the field have greatly enriched the quality of my studies and work.

Finally, I would like to express special thanks to my family for being so supportive and understanding throughout this challenging academic journey. Their prayers and thoughts allowed me to keep moving forward, even when I felt overwhelmed by both my career and academics. I cannot describe how grateful I am for that. Lastly, I would also like to thank all my close friends who also motivated me to keep moving forward towards my goals.

December 2024

A MULTI SENSOR INDOOR/OUTDOOR REAL TIME IOT HYBRID MODEL APPROACH FOR FIRE DETECTION

Jose A. Burnias, M.S.
The University of Texas at San Antonio, 2024

Supervising Professor: Jeff Prevost, Ph.D.

Fire detection systems play an essential role in identifying fire threats that could lead to property damages, risk of lives, and mass habitat losses. However, many existing solutions tend to focus on a specific use case, which affects diverse application into different settings. This study presents a hybrid fire detection model that integrates sensor data analysis, anomaly detection through the use of a Z-score statistical procedure, an AWS infrastructure, and the use of Fuzzy Logic, Random Forest Classifier (RFC), and Convolutional Neural Network (CNN) machine learning algorithms to enhance real-time fire threat identification in both indoor and outdoor scenarios. A Raspberry Pi continuously monitors collected data using temperature, gas, smoke, fire, humidity, sunlight, and GPS modules. The data is processed using Z-scores to help identify anomalies from the incoming data. When an anomaly is detected, the Raspberry Pi securely connects to a cloud-based AWS infrastructure to communicate via Message Queuing Telemetry Transport (MQTT) protocol to store sensor readings into a DynamoDB database, which then triggers a rule-based lambda function. The AWS lambda then invokes an AWS SageMaker endpoint containing a hybrid model that combines fuzzy logic and RFC to handle uncertainties and historical patterns in the data. When the hybrid model identifies a potential fire risk, a new MQTT message is transmitted back to the Raspberry Pi from the cloud to activate the connected camera module to capture an image of the surroundings. Once the image is captured,

the lambda function invokes another endpoint containing a CNN model based on the ResNet50 architecture, which categorizes images into four classes: ‘no_fire_day’, ‘fire_day’, ‘fire_indoor_night’, and ‘no_fire_indoor_night’. A web application dashboard composed of Next.js is then updated with the latest sensor readings and the captured image for users to visualize the status of the overall model in real-time. The fuzzy logic and RFC model achieved approximately 99% accuracy, while the CNN model achieved approximately 90% accuracy. Additionally, the model took approximately 774ms from database sensor data insertion to updating the dashboard, thus validating that a cloud based hybrid model approach can result effective when dealing with real-time fire threatening anomalies.

TABLE OF CONTENTS

Acknowledgements	iii
Abstract	iv
List of Tables	viii
List of Figures	ix
Chapter 1: Introduction	1
1.1 Motivation	2
1.2 Problem Statement and Solution Approach	3
1.3 Thesis Statement	4
1.4 Scope and Assumption	4
1.5 Summary of Contributions	5
1.6 Organization of the Thesis	6
Chapter 2: Background and Literature Review	7
2.1 Usage of Multi Sensor Models	7
2.2 Concept of Fuzzy Logic Systems	11
2.3 Usage of Machine Learning Techniques	13
2.4 Challenges and Limitations in Current Fire Detection Models	15
2.5 Identification of Existing Research Gaps	16
Chapter 3: Methodology	18
3.1 Sensor Layer Components	19
3.2 Edge Layer Components and Data Processing	24
3.3 Data Processing in the Cloud Layer	30
3.4 UI Layer Web based Application Dashboard	52

Chapter 4: Results	54
Chapter 5: Discussion and Future Work	61
5.1 Challenges.....	61
5.2 Recommendations for Future Work.....	62
5.3 Summary	64
Conclusion	66
Bibliography	67
Vita	

LIST OF TABLES

Table 1	Temperature Fuzzy Membership	35
Table 2	Smoke Fuzzy Membership	37
Table 3	Flame Fuzzy Membership.....	38
Table 4	Gas Fuzzy Membership	39
Table 5	Sunlight Fuzzy Membership	40
Table 6	Humidity Fuzzy Membership	41
Table 7	Fire Detected Fuzzy Membership.....	42
Table 8	Fuzzy Logic Rules	44

LIST OF FIGURES

Figure 1	Fire Detection Techniques	7
Figure 2	Fire Detection Model Architecture	18
Figure 3	BH1750 Light Sensor	19
Figure 4	Digital Output Flame Sensor	20
Figure 5	MQ-2 Sensor	21
Figure 6	DS18B20 Temperature Sensor	21
Figure 7	DHT11 Humidity Sensor	22
Figure 8	GPS Module.....	23
Figure 9	Raspberry Pi Camera Module.....	24
Figure 10	Pseudocode for fdsRaspberryPi.py	25
Figure 11	Z-Score Formula	27
Figure 12	Physical Implementation of the Sensor and Edge Layer	29
Figure 13	AWS processSensorData Lambda Function Overview	31
Figure 14	ProcessSensorData Lambda Function pseudocode.....	32
Figure 15	FL/RFC Hybrid Model Pseudocode	34
Figure 16	Temperature Membership Function Plot	36
Figure 17	Smoke Membership Function Plot.....	37
Figure 18	Flame Membership Function Plot.....	38
Figure 19	Gas Membership Function Plot	39
Figure 20	Sunlight Membership Function Plot	40
Figure 21	Humidity Membership Function Plot	41
Figure 22	Fire Detected Membership Function Plot	43

Figure 23	Random Forest Classifier Visual	46
Figure 24	CNN Model Pseudocode.....	47
Figure 25	ResNet50 based Architecture for CNN Model	49
Figure 26	Built from scratch CNN Model.....	50
Figure 27	Web Application Dashboard Interface.....	53
Figure 28	FL/RFC Results	54
Figure 29	ResNet50 CNN Model Results	55
Figure 30	ResNet50 CNN Model Results Graph	56
Figure 31	Built from Scratch CNN Model Results	57
Figure 32	Built from Scratch CNN Model Results Graph	58
Figure 33	Sensor Data Fire or Risks Query	60
Figure 34	Sensor Data No Fire or Risks Query.....	60

CHAPTER 1: INTRODUCTION

The Internet of Things (IoT), a network of physical objects (things) that are connected to the internet and have the ability to exchange data with other devices and systems, has grown over the past few years and is expected to continue growing in the upcoming years. These objects or “things” are composed of software, embedded sensors, and other technologies. As growth has increased, these devices can be found across many places such as businesses, homes, industrial applications, vehicles, etc., to name a few. Being able to connect to the internet and interact with other devices, the incorporation of cloud computing is also associated to IoT. Cloud computing provides many benefits to IoT by supporting both scalability and flexibility. To elaborate, cloud support allows the use of more resources not typically obtained in some IoT devices. These include larger storage, better performance, enhanced security, and even improved accessibility. Cloud computing usually consists of three models: Software as a Service (SaaS), Platform as a Service (PaaS), and Infrastructure as a Service (IaaS) [20]. SaaS typically provides access to software hosted in cloud servers, which give users access to software over the internet. This allows easy access and fast deployment in most scenarios. PaaS provides both hardware and software to assist in developing other software or applications. IaaS concedes with giving users access to hardware resources over the internet. A popular example of this is Amazon Web Services (AWS) [18] which provides storage, networking, and computing power.

As the use of IoT and cloud computing grows, it is no surprise that using these have become adopted into the use of fire detection systems. In fire detection, the use of sensors is largely used to monitor different aspects of fire threats. Some examples include O, O₂, CO₂, temperature, smoke, and humidity. Consequently, resources are required to be able to process and analyze incoming data, as well as be able to communicate throughout the internet to

respective points of contact. Due to these needs, some of the challenges come when processing data as many modern fire detection system approaches involve the use of machine learning algorithms, which in some cases involve the use of image and video processing. Thus, requiring a significant number of resources to be able to accomplish such tasks. Another is having access to the internet as many systems tend to be placed remotely and rely on connectivity to send signals and alerts.

By taking into account the use of fire detection systems and their need of resources to have the ability to perform such tasks, a model with its basis on IoT and cloud computing is developed to explore how fire detection can be used effectively to work in real-time.

1.1 Motivation

Fire incidents continue to be a major threat worldwide. These threats are bound to cause severe damage to residential, industrial, commercial, and natural settings rapidly when no actions are taken. As a result, this can have a significant effect on economic losses as well as loss of lives and wildlife. Being able to handle threats under diverse environmental settings can be beneficial when dealing with fire threats. With such challenge, performance metrics like inference time, robustness, reliability, and accuracy can make a fire detection model more desirable [17]. By having these characteristics, a fire detection model can make a difference in controlling any fire threat and avoid such consequences.

To date, these continue to be challenges when developing new approaches to detect fire scenarios. Many models are focused on specific conditions to bring out the best results from the designed models. Despite these approaches, some models still confront situations where resource

management could be improved or optimization of the algorithms used could be further enhanced.

These approaches and results serve as a foundation to further explore alternative approaches for handling fire threats. Additionally, the use of applying IoT, machine learning, and cloud computing technologies is intriguing on a personal level. By exploring all of these areas further, there is potential to enhance fire detection and reduce the major impact of fires universally.

1.2 Problem Statement and Solution

In literature, many models have been proposed for fire detection, however as mentioned earlier, the majority of these models tend to focus on a specific use case setting. Models that have proposed using machine learning algorithms state that further research is needed to improve accuracy, which also focus on attempting to minimizing False Negatives (FN) as well as False Positives (FP) [17]. When it comes to working with these algorithms datasets also play an important role in determining how effective they are, as there are many resources, means of collecting it, and approaches to handle missing or uncertain data.

Therefore, investigating further how fire detection models can improve on these aspects is suggested. Intuitively, a hybrid model approach combining machine learning models and adapting a method to filter out incoming data to detect anomalies is our approach. This allows early capture of data that is out of the normal and sent as input towards the machine learning models for further analysis. Through this approach we gain insight as to how hybrid models can be of benefit as well as gain understanding as to what drawbacks there may be.

1.3 Thesis Statement

This thesis proposes a hybrid approach integration of Z-Score normalization, Fuzzy Logic, Random Forest Classifier (RFC), AWS infrastructure, and a ResNet50 based Convolutional Neural Network (CNN) for real-time multi sensor data and image processing. The proposed solution addresses the critical need to ensure accurate and timely identification of fire threats while minimizing false alarms and enhancing detection reliability across diverse environments. Leveraging cloud integration through AWS ensures scalability, robust processing capabilities, and efficient resource utilization for resource constrained models. This effectively creates a synergy between edge and cloud-based processing, achieving a balance of adaptability, efficiency, and accuracy in fire detection models.

1.4 Scope and Assumption

The scope of this study surrounds the development of a hybrid fire detection model using machine learning techniques and statistical calculations for analyzing multi sensor data and images to enhance detection accuracy and reduce the detection of anomalies that may lead to false alarms. The work focuses on an application-based model that has the ability to work in diverse settings. Particularly, within indoor and outdoor scenarios considering variations in lighting conditions and sensor readings. The work utilizes a predefined dataset for training based off open-source fire and no fire scenarios. To simulate real-time working conditions, there were certain limitations such as the restriction to create a fire in lab and research facilities. As a result, the use of artificial data modeled on historical records was employed to simulate real-time sensor data every couple of seconds. Some of the several assumptions that were considered under this study, include:

- The accurate calibration of sensors and proper installation/maintenance of sensors and equipment done by end users.
- The assumption that no service interruptions to AWS or the cloud-based services used were impacted.
- That the edge device and all power dependent components were fully powered and had stable internet connectivity for calculating inference time.
- Limited image data diversity which may not represent all real-world fire scenarios resulting in unfamiliar situations limiting the model's ability.
- Access to the model and AWS infrastructure is only accessible to enabled individuals.

This leaves unauthorized individuals out of scope due to security related aspects.

These assumptions create a controlled context that is used towards evaluating the performance of the system and determining limitations in areas that could be future areas of research scope.

1.5 Summary of Contributions

This study analyzes some of the models and systems proposed in recent literature to determine challenges and criteria to further evaluate them against a hybrid modeled approach. The hybrid model makes use of state-of-the-art machine learning algorithms to work collaboratively with each other and deliver improvements on performance. This study also implements a cloud-based architecture solution that makes use of an AWS infrastructure to make use of cloud computing resources in order to provide enhancements in performance, scalability, and reliability as a solution. This creates a supportive and flexible environment for real-time data processing and inference by handling complex operations where data may come in large volumes. As part of monitoring the status of the model, a dashboard is also introduced to provide

a comprehensive view of real-time data feed through the use of a Next.js framework. Through this contribution, the predictions of the hybrid model are available to users immediately, which offers an insight without requiring any technical effort for simplicity.

1.6 Organization of the Thesis

The rest of the thesis is organized as follows. Chapter 2 provides a brief background of the topics that form the basis of this thesis. This covers relevant work related to the most recent literatures. Chapter 3 discusses the methodology of how the hybrid model works and what steps were taken to create the working model. It includes discussion on how z-scores are implemented into the model as well as how fuzzy logic, RFC, and CNN work together as a whole under the AWS infrastructure. Chapter 4 goes over some of the results that were obtained after the hybrid model was created. This chapter also covers some of the criteria and steps taken to develop a testing scenario for the data collected. Chapter 5 then serves as discussion for some of the challenges that were faced throughout the process and also provides a comprehensive insight of what future work is recommended to further enhance the proposed model. Ultimately, highlighting potential points of interest for future research. Finally, the last section of this paper provides a conclusion that summarizes the work on this thesis.

CHAPTER 2: BACKGROUND AND LITERATURE REVIEW

This chapter discusses recently related work along with concepts and required background that form the foundation towards this thesis. First, it gives emphasis to the usage of multi sensor models in fire detection. Furthermore, it examines the concept of fuzzy logic and its application in the development of combinational models. Discussion on the usage of varying machine learning techniques and their observations towards performance and accuracy is also highlighted. Finally, the challenges and limitations in current fire detection models along with some of the existing research gaps is also discussed.

2.1 Usage of Multi Sensor Models

In the realm of IoT based fire detection systems, the use of sensors is widely used to take an effective approach in enhancing the accuracy and reliability of predicting conditions. Some of the most common form of sensors and techniques include: smoke, flame, gas, and ambient temperature. [4]

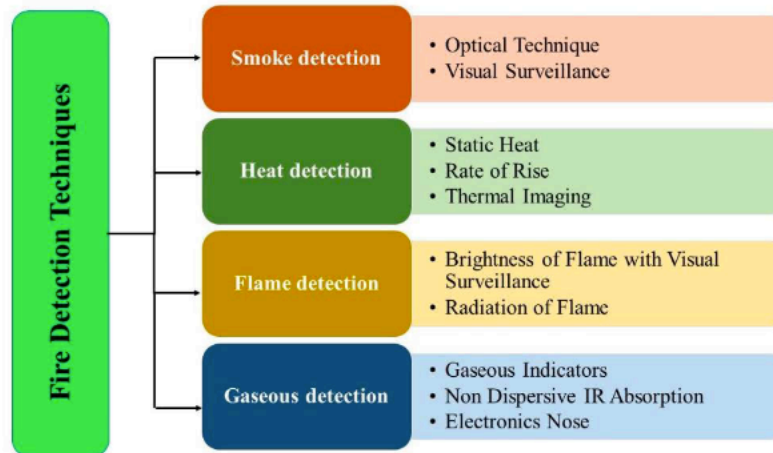


Figure 1: Fire Detection Techniques [4]

Figure 1 shows the different fire detection techniques that sensors can aid with. Through the use of sensors, existing fire sensing systems are able to convert physical parameters into suitable electronic signals that can be processed. The processing techniques usually consist of machine learning algorithms and include an alarm or notification-based signal to indicate a fire. To converse further how smoke sensors are designed, they can be divided into two categories: optical and visual surveillance. Optical based smoke sensors typically use light or electromagnetic fallout when particles in the path of a receiver are influenced. However, some of the downsides to this type of smoke sensor include an increase in false positives and alerts due to the impact dust or even aerosols can put out. As this is one of the most common forms of smoke detection it is highly suggested to employ the use of multi sensing approaches to serve as support to the optical sensor's sensitivity. [4] The other form of smoke detection involves visual surveillance, which can be a modern way of monitoring smoke signals as it uses cameras to extract image features such as color, texture, and movement analysis.

Flame sensors are commonly used to detect the presence of flames. This type of sensor also has several categories that may assist in detecting them. These categories include: Infrared (IR), Ultraviolet (UV), and visible light or multi-wavelength flame sensors. The application of these involves being able to measure brightness and radiation of a flame. UV sensors are heavily used to detect brightness while IR sensors are used more towards radiation. On the other hand, multi-wavelength sensors calculate material temperature, radiation, and existence of a flame. In some situations, it is chosen to combine some of these to reduce false alarms due to the unique specialty of each different flame sensor type and because of the close proximity requirement the sensors have to be to a flame. In more modern work, just like in smoke, the use of visual

surveillance has been widely adopted to extract flame features like color, patterns, and even region surroundings [4].

The use of ambient temperature sensors is potentially one of the most important characteristics in fire detection. This involves the sensing of changes in temperature related to heat. There are several ways this can be achieved. The categories include: static heat detectors, rate of the rise detectors, and thermal imaging. Static heat detectors are quite straightforward since they react to heat by melting or shorting when the sensor is exposed to a predefined temperature. A major disadvantage of this type of sensor is that it is generally a one-time use, which requires replacing the sensor since it fails to function further. Rate of the rise detectors behave differently; they are developed to detect fast changes in environmental temperature over a period of time. As a result, this type of functionality can also lead to false alarms or anomalies. The last type of heat detection associates thermal imaging to infrared cameras with the ability to visualize patterns of radiation as a means of feature extraction and analysis [4].

Finally, gas detection also plays an important role in fire detection techniques since this type of sensor focuses on detecting combustion of organic elements as a result of flames. This incorporates the production of gases such as CO and CO₂ which are some of the most common forms of gas indicators when dealing with flames. To achieve the detection of these gaseous compounds, gas sensors include: non dispersive IR and electrochemical. Non dispersive IR is based on optical characteristics like change in color while electrochemical focuses on the electrochemical reaction of the sensing material [4].

Due to the diverse options of identifying such features and the uncertainty some of these sensors can deliver as part of their strengths and weaknesses, the idea of combining multi sensor systems with image recognition algorithms to assist the sensor data is proposed. By using

different classification methods along with combined sensor data more efficiency and accuracy can be achieved [2]. Literature also dictates that algorithms that measure for time series data to capture different shapes and temporal information can be used to identify early stages of a fire through the use of multi sensor systems [3]. Some popular approaches to handle uncertainty or noise in sensor data is by associating data fusion through the use of filters like adaptive distributed Kalman (ADFK) or even fuzzy logic [6]. As mentioned earlier, some of the commonly used fire detection sensors may result in inconsistency or correlation in data due to some of their weaknesses. Therefore, it is crucial to improve data quality by preprocessing data to detect outliers, which may point to anormal data points caused by sensor failure or extreme conditions [6]. From these applications and perspectives, the idea of a hybrid modeled approach is inferred to improve reliability of a fire detection model. In fact, there have been instances of work that have studied the use of a hybrid machine learning algorithm using a maximum averaging voting classifier, which uses four classifiers: logistic regression, support vector machine (SVM), decision tree, and naïve bayes classifiers to achieve better prediction and improved robustness [12]. Fundamentally, by combining various learning classifiers prediction accuracy can be improved over the use of a single classifier model.

Additionally, the use of sensors is not limited to just the ones discussed so far. Many other sensors such as humidity, sunlight, air quality, and GPS modules can also be used to help analyze environmental data of where a fire detection system may be placed. By expanding the options further, the capacity to analyze and predict more characteristics of potential fire risks can be enhanced. In [10, 14] the authors also make use of protocols such as MQTT to transmit sensor data over cloud services to trigger notifications and display the data in a user interface, which is a common approach in many fire detection systems to handle sensor data across proper parties.

2.2 Concept of Fuzzy Logic Systems

The concept of fuzzy logic extends the idea of Boolean logic in the sense of partial truth. While the traditional concept represents truth or false, the values in fuzzy logic represent degrees between 0 and 1 [7]. This allows reasoning for approximate or imprecise data that make it useful in scenarios where binary logic does not usually work. Some of these examples reside in control systems, decision making, and pattern recognition. In order to handle those type of scenarios, fuzzy logic is composed of several key components that make it possible. The basis of fuzzy logic Fuzzy consists on fuzzy sets, which express an element's membership by a degree of truth in an interval of 0 to 1. An example of a fuzzy set can be in temperature where "hot" ranges from a specific lower and upper bound value. The closer to the lower bound of the fuzzy set, then the lower the degree of membership it has on the fuzzy set, while the closer to the higher bound value gives it a stronger membership. These types of relationships can be defined within a fuzzy membership function that declare how each point in the set are mapped between 0 and 1. Membership functions usually can include triangular, trapezoidal, and gaussian membership types, which help quantify the fuzzy concepts in fuzzy sets that can represent something like "cold", "warm", or "hot" [11]. The type of memberships can also be useful in identifying overlaps between these functions since it is possible for sets to have a sense of belonging between multiple sets. In order to calculate some of these overlaps, the process of fuzzification applies. This is the process in which crisp input values from the fuzzy sets apply to the membership functions to give them that sense of belonging. Finally, by using the crisp values from each membership function in the system, a set of fuzzy rules are applied in the form of if-then statements that help represent the relationships between fuzzy inputs and outputs [6]. These rules practically drive the decision making that create an inference system. An output is

produced representing a final decision of the system, which then goes through a process called defuzzification. This process is the last step in fuzzy logic, which exists to convert back the final output to a crisp value that represents a numerical output related to a temperature or risk value [7].

There are several advantages with using fuzzy logic in systems. Some of these are the ability to handle uncertainty, the flexibility to mathematically define complex systems through the use of linguistic expressions easy to understand, transparency to validate and adjust the defined rules, and the potential to work well with incomplete or imprecise data, thus not requiring a precise system model. With these advantages there are also some challenges like increase in rule complexity if there are changes to the variable inputs, which make the system harder to manage. This can sometimes cause expensive computational costs due to the exponential growth of variables.

In fire detection systems some of these fuzzy logic concepts are valuable, especially when handling real world sensor measurements due to the many environmental factors that can lead to vagueness, uncertainty, and imprecision. Therefore, some studies like [5] have implemented fuzzy logic to handle multi sensor inputs in indoor scenarios to calculate condition levels of a room. By making use of fuzzy logic, they found that it helped reduce data storage and computation at the fusion center of the system. Using fuzzy logic also enhanced sensitivity by integrating data from multiple sensors, thereby increasing the detection coverage and minimizing the chance of false alarms. This is also supported by the work in [7] which found that unwanted fire alarms could be reduced by approximately 80% through the use of fuzzy logic on an IoT based fire detection system that was tested using field data from a period of 5 years. Other types of research surround the idea of implementing fuzzy logic along with other algorithms to

perform data fusion. These approaches discuss the processing of multi sensor data through adaptive filters and logical reasoning algorithms along with fuzzy logic [6, 11, 15]. By combining these options, fused data had significant accuracy improvement and refined results were obtained as a system overall.

2.3 Usage of Machine Learning Techniques

Machine learning has grown to be an important part of problem solving in the past few years. Some of its applications have been adopted across many domains since it has the ability to analyze data, identify data patterns, and make predictions. One of those domains is modern fire detection, where sensor data, image analysis, and video processing are leveraged to make timely and accurate decisions. Literature states that typical techniques in sensory systems are Decision Trees (DTs) and Random Forests (RFs), Support Vector Machines (SVMs), K-Nearest Neighbor (KNN) and K-Means Clustering, Neural Networks (NNs), Convolutional Neural Networks (CNNs), Long Short-Term Memory (LSTM), and Gated Recurrent Units (GRU) [17]. While most of them are used in different scenarios, they are the most commonly used in modern systems depending on layout and settings. For example, in [1] the use of machine learning supports real time sensor technology to predict and detect unsafe conditions in the fireworks industry. Through the use of the KNN algorithm historical data can be analyzed to predict safety hazards and produce a live stream to further monitor the hazards foreseen. Other applications such as in [2, 13] propose combining multi sensor detection systems with image recognition. The majority of image processing techniques make use of popular algorithms like CNN to extract image features for decision making. Even though image recognition is widely used, the use of video processing has also seen large growth. In [9] the authors make use of video processing by

uniquely combining machine learning models to extract image features and environmental audio for classifying forest fire threats on low power IoT devices. However, it was mentioned that the hardware-imposed limitations due to image resizing and that without the audio NN, the picture CNN caused too many false positives. There have also been some studies [8, 12] related to the use of decision trees or RFC to develop more reliable prediction systems due to the ability these impose when handling high dimensional feature spaces through the use of several sensors. Other advantages include: high accuracy, robustness, versatility, reduces overfitting, feature importance, efficient parallel processing, natural handling of missing values, ensemble learning benefits, and no need for feature scaling [8]. They also mention how combining multiple classifiers obtain enhanced results, as well as make RFC comparison to other classifiers like KNN, where RFC achieved better results. This is supported by the work done in [16] where the authors find that KNN, RFs, and DTs give the most accurate results when used with multi sensor systems across different scenarios, but RF showing better results in more environments than KNN.

Essentially, machine learning has revolutionized modern fire detection systems enabling more accurate and reliable responses to potential threats. By leveraging images, sensor data, video, and advanced algorithms that combine these approaches newer systems have proven to reduce fire related risks across several existing domains where fire can be threatening. As a result, challenges that still require further research to reach the full potential of these algorithms is still in progress. In addition, many of these studies mention how algorithm optimization is crucial and how false positives or false negatives can be reduced by the use of hybrid models that complement each other [17].

2.4 Challenges and Limitations in Current Fire Detection Models

Having discussed some of the current applications in modern fire detection systems, there are also ongoing challenges and limitations that need further research. For instance, false positives and false negatives occur often from sudden changes in sensors due to environmental factors, or even low visibility and overlapping features in images. These can trigger false alarms unnecessarily and are a struggle fire detection models currently face. Data quality and limited availability is also a challenge since machine learning models rely heavily on training data to perform accordingly. This also contributes to the call for further research on algorithm optimization to aid in minimizing false alarms.

As with false negatives and false positives, environmental variability also affects many modern systems. This comes from the diverse conditions that fire detection systems must contend both indoors and outdoors. Weather can play an important role in the accuracy of analytics, but failure in sensors can also have a major impact. Therefore, different ways of handling these circumstances are a work in progress. In addition, due to the remoteness of some outdoor setups, energy efficiency is also a resource that systems must manage, especially when dealing with continuous monitoring or processing large datasets that may cause high power consumption.

Fire detection system studies also explore the concept of latency [19] and response time since complex systems may introduce delays. This can be a major challenge, particularly in real time detection systems where decisions can be critical in high stake situations. The dependance of network connectivity can also affect this metric. Ultimately, these are some of the current and most crucial challenges researchers experience with fire detection models.

2.5 Identification of Existing Research Gaps

Despite significant research in fire detection systems as the ones discussed up to this point, notable research gaps still exist. Touching back basis on environmental factors, many of the ongoing research focuses on specific domains such as industrial [1], indoors [3, 5, 15], or outdoor settings [9, 21]. There's limited research on validating hybrid models in diverse environments, which is something this thesis makes an effort in addressing by incorporating contextual sensor data, image classification, and flexible decision-making frameworks in both indoor and outdoor conditions.

Current research also has major focus on detection accuracy and machine learning performance metrics. While these are definitely crucial in fire detection systems, many ongoing research fails to prioritize or measure response speed. This is essential for real time detection systems that rely on reacting in a timely manner to potential fire hazards, where delays in detection or communication can result in damages and safety risks. Ensuring fast and reliable decision-making features are critical to mitigating threats. This thesis takes this into account by evaluating response times across each part of the decision-making process.

Other gaps in current research surround the scalability and deployment of models in low resource environments. While most research focuses on high performance systems, they often overlook constraints that make them less accessible and practical for widespread use. Addressing this gap is critical to ensure that fire detection systems can be effectively deployed in diverse settings regardless of the available resources. To acknowledge this gap, this thesis makes use of cloud computing services such as AWS to implement a hybrid approach that balances local processing on resource constrained devices like the Raspberry Pi, with the scalability and computational power of the cloud. This helps offload complex operations such as image and

sensor processing where the system ensures real time decision making without overwhelming the local hardware. In addition, it also helps ensure that the fire detection system is accessible throughout diverse environments.

In the use of image-based fire detection ongoing research typically focuses on binary classification, where it evaluates fire and no fire scenarios. Gaps exist in exploring how images can specifically include diverse fire scenarios such as indoor, outdoor, day, night, etc. This thesis explores the use of multi-class classification to add granularity and improve decision making for fire responses. The approach also serves as motivation to explore the optimization of CNN models since it is an ongoing research area aimed at improving the efficiency and accuracy of diverse settings.

The next chapter discusses the methodology of the fire detection model proposed in this thesis. It also serves as a continuation of how the architecture and approach acknowledge some of the discussed literatures and their gaps.

CHAPTER 3: METHODOLOGY

This chapter presents the methodology and architecture used to develop and implement the proposed hybrid fire detection model. The model is composed of four layers: sensor, edge, cloud, and UI layer. Each layer consists of different components that handle different aspects of the data collection, processing, decision-making, and visualization pipeline. Together they work in a coordinated manner to ensure the system's accuracy, responsiveness, scalability, and usability. Each section will go into detail of the mechanisms happening within each layer to accomplish these results. Figure 2 shows in detail what each layer is composed of and how they connect to each other to create the architecture of the fire detection model.

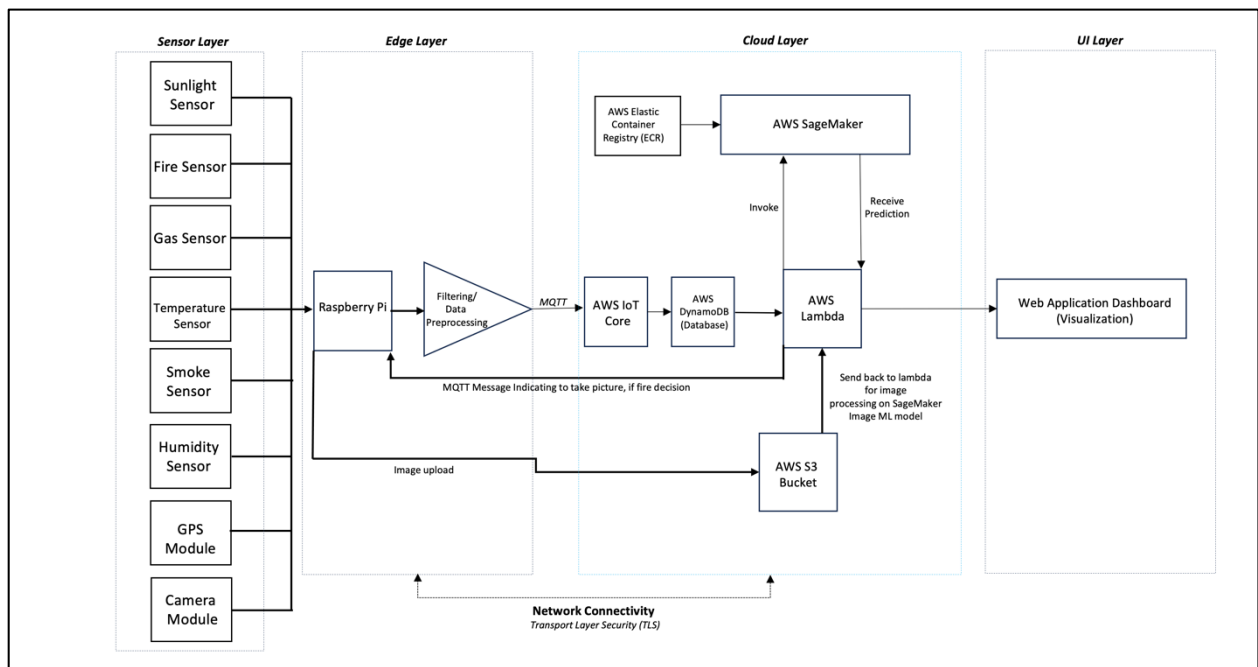


Figure 2: Fire Detection Model Architecture

3.1 Sensor Layer Components

The role of the sensor layer is focused on collecting environmental data in real-time. It forms the foundation of the whole architecture by providing the raw data that is used for further processing in the other layers. As illustrated in Figure 2, the sensor layer is composed of eight distinct sensors and modules. These are: sunlight, flame, gas, temperature, smoke, and humidity sensors, along with GPS and camera modules.

A sunlight sensor is used in this model to measure the intensity of the ambient light in the environment. This data is expressed in the lux, a standard unit for illuminance. Its main purpose is to help determine whether the monitored environment is in daylight or low light conditions, which are used to categorize into “day” and “indoor_night” representations. Daylight instances usually give off high lux values, while night and indoor domains give off less. As part of the sensor calibration, it was determined that any readings of 10000 or more lux would categorize as “day” to indicate that a strong presence of light intensity is present. Anything less than 10000 was classified as “indoor_night” to represent indoor and night environments. Figure 3 illustrates the sunlight sensor used to measure light intensity in the model.

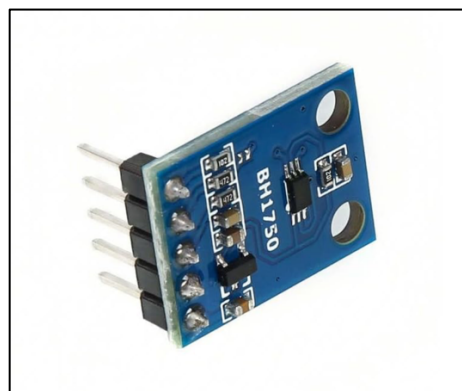


Figure 3: BH1750 Light Sensor

A flame sensor was used in the model to help detect the presence of fire, which may indicate potential fire outbreaks. The chosen sensor is based off infrared technology that helps detect radiation emitted by flames. A digital signal indicating the presence of fire based off voltage output was used as measurement for this sensor. As a result, a binary based signal is represented through the voltage output to indicate 0 (LOW) or 1 (HIGH). Figure 4 shows a visual representation of the flame sensor used for this model.

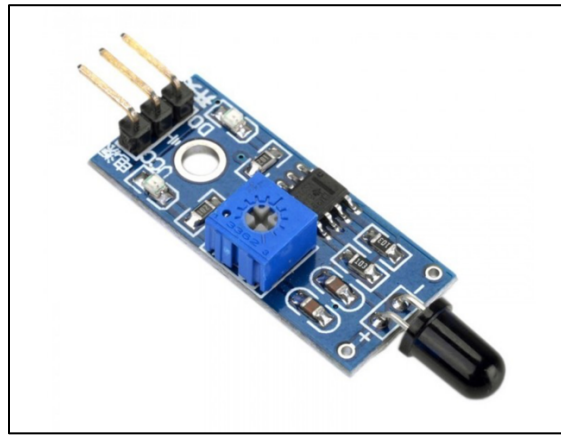


Figure 4: Digital Output Flame Sensor

The gas sensor plays an important role in this model by helping detect concentration of combustible gases and byproducts of combustion that serve as key indicators to potential fire hazards. In particular, the gas sensor in this model focuses on gas concentrations of carbon monoxide (CO), methane, propane, alcohol, and hydrogen. Most of these gases may not be immediately detectable through visual sensors like smoke and flame, so it enhances the sensory of environmental hazards. The measurements used for this sensor were in units of parts-per-million (ppm), which measures gas concentrations. Further discussion on the ranges used will be discussed in the next sections. Figure 5 shows the MQ-2 sensor used to detect gas concentrations within the hybrid model.



Figure 5: MQ-2 Sensor

Temperature is a fundamental feature in fire detection providing data to identify abnormal heat conditions that may indicate a fire. Since flames often start off with an increase in temperature, the use of a temperature sensor is adopted. The sensor chosen for this model has a wide range temperature digital output capacity (-55°C to 125°C) to ensure accurate and reliable measurements, especially in handling environments with high temperatures. These values are converted to Fahrenheit, with no significant effect to the functionality of the sensor itself. Figure 6 illustrates the sensor used in the model.

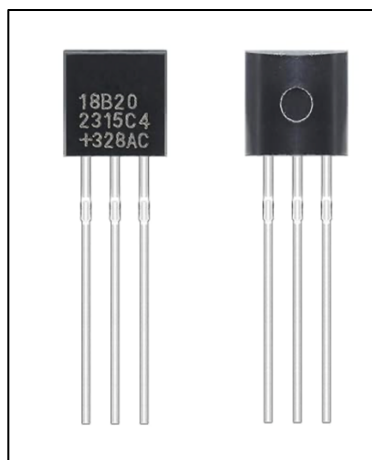


Figure 6: DS18B20 Temperature Sensor

In fire detection, smoke is also an important feature to monitor for, as these types of sensors provide early warnings of smoke particles that could indicate early signs of combustion. For this model, an MQ series general purpose smoke sensor was used, which is comparable to the one used for gas in Figure 5. Relatively, the units measured by this sensor also correlate to the parts per million (ppm) unit.

The last sensor used as part of this model is used for humidity. This type of sensor measures moisture in the air, which is critical in fire detection since it influences on both ignition and spread of fires when conditions are dry. By including this sensor in the model, a broader perspective of the environmental conditions can be observed to evaluate risks by complementing other sensors like temperature, gas, and smoke. Figure 7 shows a visual of the sensor, which has a digital output signaling the relative humidity in terms of percentage. Therefore, a low relative humidity signals drier conditions, while a higher relative humidity value points towards high concentrations of moisture.

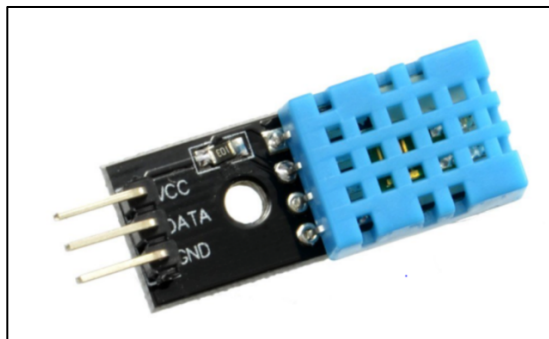


Figure 7: DHT11 Humidity Sensor

One of the modules used as part of the model's sensor layer is the GPS module. This supports the fire detection system by providing the coordinates in terms of latitude, longitude, and altitude to allow responders to locate the fire threats precisely. It can also be used to assist in tracking historical fire events based off the historical data. Theoretically, this data could also be used to integrate a visual representation of the coordinates using platforms like google maps, however, this was not part of this model's work. By including this module in the model, real-time data can contain further details that can benefit tracking down the potential threat to be dealt with promptly. Figure 8 illustrates the GPS based module for this model.

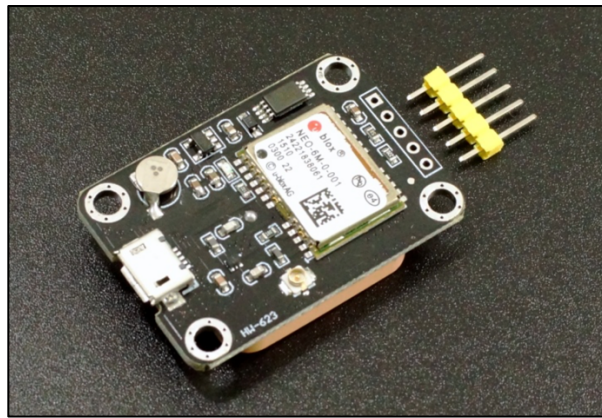


Figure 8: GPS Module

The last module as part of the sensor layer is a camera module. Figure 9 shows a visual representation of how the camera looks like. The decision to include a camera module in this model was to act as an additional layer of confirmation towards fire threats. This particular camera module has a resolution of approximately 12 megapixels which produces sharp images that can be used for further analysis through the use of machine learning algorithms to extract image features. In this model, the camera acts as a middle man between the edge and cloud layer,

which relies on communication protocols like MQTT to determine whether an image should be captured or not. This will be further discussed in the next sections, but the addition of the camera module adds value to the decision-making process of the model.



Figure 9: Raspberry Pi Camera Module

All the sensors and modules discussed in this section are directly connected to a Raspberry Pi, which is part of the edge layer and connects the sensor layer into it. Within the edge layer a lot of the initial filtering and pre-processing of the raw data produced by the sensors and modules occurs. The next section will discuss this process further.

3.2 Edge Layer Components and Data Processing

Following the pipeline from the architecture in Figure 2, the edge layer is composed of a Raspberry Pi that supports the sensor layer by handling all of the pre-processing and filtering aspect of the data before being sent to the cloud layer for decision making. All of this is handled by a python script that is ran within the Raspberry Pi. Figure 10 shows the pseudocode for the script.

```

1. Initialize Environment and Libraries
a. Import necessary libraries (e.g., numpy, paho-mqtt, boto3, etc.).
b. Load environment variables for MQTT broker, port, topics, and AWS credentials.
c. Initialize global variables:
    - Sensor data lists
    - Threshold values for temperature, humidity, gas, smoke, etc.
    - Event for MQTT message reception.

2. Define Utility Functions
a. `celsius_to_fahrenheit(celsius)`: Convert Celsius to Fahrenheit.
b. `format_timestamp()`: Generate formatted timestamp (YYYY-MM-DD HH:MM:SS).
c. `create_partition_key(sensorid)`: Create a unique key using sensor ID and timestamp.
d. `upload_to_s3(file_name, bucket_name, object_name)`: Upload a file to an S3 bucket, handling exceptions.
e. `generateSensorData()`: Simulate random sensor data for temperature, flame, humidity, sunlight, gas, smoke, and GPS.
f. `calculate_z_score(data_list)`: Compute the Z-score for the most recent data point.
g. Threshold-checking functions:
    - `last_n_values_exceeding_threshold(sensor_data, threshold, n)`
    - `last_n_values_below_threshold(sensor_data, threshold, n)`
    - `last_n_values_sum(sensor_data, n)`

3. Define MQTT Functions
a. `publish_sensor_data(sensorData)`: Publish sensor data to MQTT topic.
b. `subscribe_message(client, userdata, message)`: Handle received MQTT messages and set event flag.

4. Establish MQTT Connection
a. Configure MQTT client with TLS and credentials.
b. Attempt to connect to the MQTT broker; exit on failure.

5. Main Loop: Real-Time Data Simulation
a. Generate random sensor data using `generateSensorData()`.
b. Append data to respective lists and maintain the last `DATA_SIZE` readings.
c. Calculate Z-scores for temperature, humidity, sunlight, gas, and smoke data.
d. Count the number of sensors exceeding Z-score thresholds.

6. Anomaly Detection
a. Check if recent values exceed or fall below thresholds using helper functions.
b. Calculate the number of anomalies detected by Z-scores and threshold checks.

7. Publish and Respond to Anomalies
a. If anomalies exceed the defined count, publish sensor data to MQTT topic.
b. Subscribe to the MQTT topic for responses.
c. Wait for a message event to be triggered.

8. Handle Image Capture on Command
a. If the response message is `take_picture`:
    - Configure the Raspberry Pi camera and capture an image.
    - Save the image locally and upload it to the S3 bucket.
    - Stop the camera and clear the event for the next message.
b. If no image capture is required, skip this step.

9. Log Sensor Data
a. Print sensor data and Z-scores for monitoring purposes.

10. Repeat Process
a. Continue the loop to simulate real-time data processing.
b. Handle exceptions to ensure robustness.

11. Exit Gracefully
a. On script termination, clean up resources and exit safely.

```

Figure 10: Pseudocode for fdsRaspberryPi.py

The script begins by initializing all the required libraries to perform operations and functionalities. For example, NumPy is used to generate data based off historical values to simulate real-time data. This is mainly due to the limitations encountered when testing the model, since a real-world scenario where flames are present was not feasible. Other libraries such as paho-mqtt and boto3 serve as support for actions like connecting to AWS and transmitting sensor data through MQTT. Along with the libraries, environmental variables that contain connection details are also declared. Since some of these contain confidential information, a separate .env file was created to retrieve them as contained environmental variables. This approach was taken to acknowledge security risks, such as pushing credentials into public repositories. In addition, global variables are also included as part of initialization to declare array lists, constant values, and threading events that are used across the script.

The script is then followed with all the utility function definitions. The purpose of these functions was to facilitate complex operations. These included:

- Converting temperature values from Celsius to Fahrenheit
- Formatting the timestamp to display specific details
- Creating a unique sensor id to distinguish all incoming sensor data
- Uploading files to AWS S3 to store logs and images captured by the camera module
- The function that generates artificial data for each sensor and formats it to the respective data types
- Calculating the Z-Score for the most recent data, which is crucial to filtering and data pre-processing
- Additional calculations that determined whether the incoming data exceeded threshold values

- MQTT functions to publish and subscribe to topics

By having these functions available, the script was kept tidy and made it easier to call on operations at demand.

Once everything was declared, the script then followed logic to establish secure MQTT connectivity. This was accomplished securely through the use of TLS, which used certificate and access keys that were generated from the AWS sandbox. By implementing a secure connection, all sensor data transmitted could be safely sent through an established network connection from the Raspberry Pi to AWS cloud services.

As the main portion of the script, a while loop is actively fetching sensor data every 10 seconds. Within this loop the function to generate sensor data is invoked and appends the sensor readings to the defined lists that pertain to each different sensor in use. Per this work, these lists keep track of the last 10 sensor readings (Approximately last 2 minutes of data). The value is stored as a constant and could be modified, however, to analyze a broad perspective of data, 10 was depicted as the best choice. Once, the lists had sufficient data, the Z-Score calculations were implemented. Figure 11 demonstrates the formula for calculating Z-Score.

$$Z = \frac{X - \mu}{\sigma}$$

Figure 11: Z-Score Formula

Z-Score is a statistical measurement to describe how far a data point is away from the mean of the dataset in terms of standard deviations. Represented as follows:

- Z: Z-score

- X : The data point (temperature reading)
- μ : Mean
- σ : Standard deviation of the data

This statistical analysis was primarily chosen to eliminate the weight assignment of sensors over others, as encountered by the work in [3]. Normalization of the data was essential in this work since each sensor had different scales of measurement. The use of Z-Score in fire detection has the ability to detect unusual sensor readings, classified as outliers or anomalies. These unusual readings were derived from unusual fluctuations of data or representations of sensor failures. In this thesis, a Z-Score representing 2.6 or greater gave the best results in terms of effectively detecting only risks of fire or actual fire scenarios. This is equivalent to approximately a 99% confidence degree of certainty. More will be discussed in the results section of this thesis. In addition to the Z-Score logic, a backup threshold implementation of incoming sensor data was included. This logic focused on checking if the last n values, where $n = 7$, exceeded their respective threshold, confirming potential threats. It was mainly added to remediate the possibility of the Z-Score implementation to not reach the specified Z-Score if the last 10 values were high enough in the sensor's upper range limit. Both of the Z-Score implementation and the threshold-based backup send data through the MQTT protocol to AWS if half or more of the sensors detect an anomaly, thereby confirming multiple signals that a fire threat is present.

The script also contains logic to handle image capture on demand. At this point of the implementation data has already been sent to the cloud layer via MQTT, so the Raspberry Pi remains on standby awaiting a response based off the decision-making process algorithms part of the cloud layer. When a response is received to capture an image, the script signals the Raspberry

Pi to trigger the attached camera module and capture an image of the surroundings (“take_picture” message response via topic). The logic uses the defined function that uploads data to an AWS S3 bucket, where the 296 x 296 image is stored. On the contrary, if the cloud layer signals the edge layer that no image capture is needed, the script signals the Raspberry Pi to print out the sensor data for the specific time frame into the console and to repeat the process, while handling all the necessary exceptions gracefully for robustness.

To briefly reiterate, the edge layer is composed of the Raspberry Pi, which works based off a Python script that handles incoming sensor data and processes it through the use a Z-Score statistical implementation. Based off anomaly detections, sensor data is transmitted via MQTT into the cloud layer for further processing in the decision-making process. If signaled, the script also handles communication from the cloud layer to capture images that assist in the final decision layer. The process is then repeated every 10 seconds to further evaluate incoming data in efforts of detecting anomalies before being sent out for further analysis.

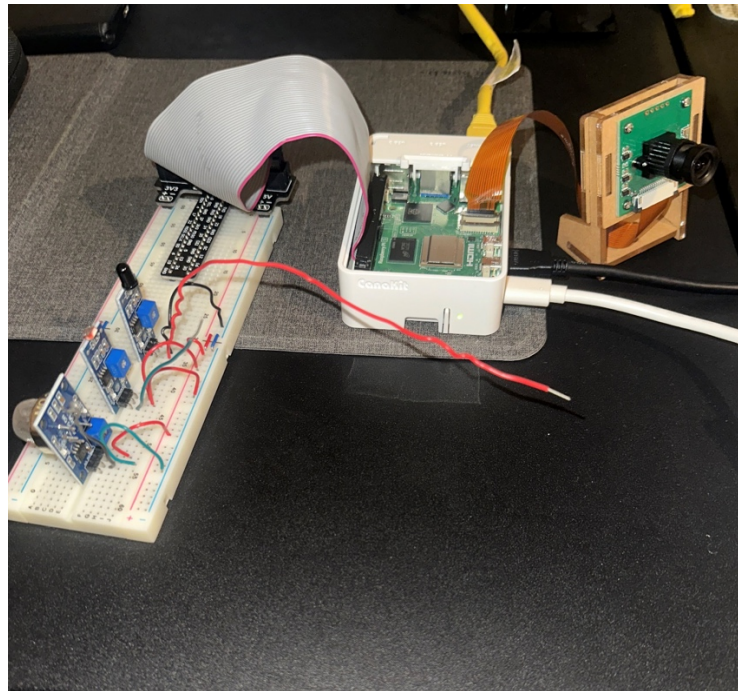


Figure 12: Physical Implementation of the Sensor and Edge Layer

Figure 12 provides a visual representation of the physical implementation of both the sensor and edge layers, as outlined in the architecture from Figure 2.

3.3 Data Processing in the Cloud Layer

The third layer in the proposed architecture model in Figure 2 consists of cloud-based services within an AWS infrastructure. This layer is connected to the edge layer through network connectivity, which uses the Transport Layer Security (TLS) protocol. In order to establish this connection, the Raspberry Pi must be setup to connect as an IoT device within AWS IoT Core. The process involves creating a “thing” under the managed devices of the AWS IoT Core sandbox. By creating the Raspberry Pi as a “thing” a device certificate, private key, and Amazon root CA are generated. In addition, IoT policies giving permissions to the Raspberry Pi must be created and attached to allow the performance of specific actions. This provides restrictions and permissions to the Raspberry Pi based off the user needs. Once everything is setup, the certificate, key, and root CA files must be placed inside the Raspberry Pi folder structure, typically in the location of the project or script that will be using these elements. Through these steps, the Raspberry Pi is prepared to connect securely to AWS IoT Core via TLS, as shortly mentioned in [20]. As part of the policies, the ability to use all IoT Core resources such as the MQTT client were given to the Raspberry Pi. Given these policies, the Raspberry Pi can securely publish and subscribe to MQTT topics, which is primarily used to publish sensor data.

Following the AWS IoT core setup, a DynamoDB table was also created. The sensorData table composed of a primary key, sort key, and the different sensor fields, was used to store the real-time data that was sent from the sensor and edge layers. Through AWS IoT core, a rule was created to automatically store incoming MQTT messages into the sensorData table. The rule,

was basically setup to map the incoming JSON data from the MQTT message into each of the table's columns through an insert statement. The rule was setup to have permission to write to the sensorData table through an IAM role. Essentially, after setting up all the necessary permissions, roles, and elements all sensor data sent through MQTT would automatically be available to the model's database.

Similar to the process of creating roles, policies, and required accessibility components, an AWS lambda function called processSensorData was created. This lambda function, however was setup to trigger based off a DynamoDB stream. The purpose of this stream was to capture changes in the database. For example, when a new item was written into the sensorData table, the lambda function was triggered to process the data. As part of the lambda function, permissions into other services like S3 and CloudWatch were given to allow accessibility to additional resources necessary for decision making and for logging to trace back errors or event details. Figure 13 shows the processSensorData function overview of how it is triggered.

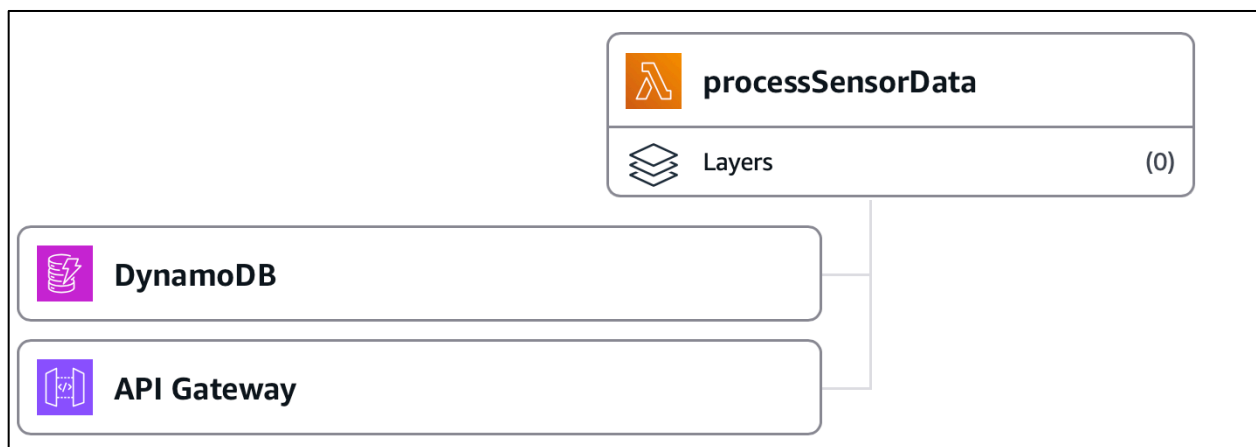


Figure 13: AWS processSensorData Lambda Function Overview

Internally to the processSensorData lambda, the basis of the model's decision-making process is handled. Figure 14 shows the pseudocode of how the lambda works in more detail.

```
1. Initialize AWS SDK clients:
   - S3 client for generating signed URLs.
   - IoT client for sending MQTT messages.
   - DynamoDB client for reading/writing sensor data.
   - SageMaker runtime client for invoking ML endpoints.

2. Define constants for:
   - MQTT broker endpoint.
   - MQTT topic to publish messages to Raspberry Pi.
   - S3 bucket and image details.
   - Predictive hazard scenarios (such as fire or risk predictions).

3. For each record in the DynamoDB stream event:
   a. Check if the event is an INSERT (new data):
      - Retrieve the new sensor data from the DynamoDB record.

   b. Prepare the data to be sent to the FL/RFC model:
      - Serialize relevant sensor data for the FL/RFC model (e.g., temperature, smoke, flame, etc.).

   c. Invoke the FL/RFC model (SageMaker endpoint):
      - Send the sensor data to the FL/RFC model.
      - Capture the prediction result (flRFCPrediction).

   d. If the FL/RFC model predicts a hazard (e.g., "fire" or "risk"):
      - Publish an MQTT message to Raspberry Pi to take a picture.

   e. If the MQTT message is successfully published:
      - Wait for the picture to be captured.
      - Generate a pre-signed URL for the image stored in S3.

   f. Prepare the image URL and send it to the CNN model (SageMaker endpoint):
      - Prepare the input data for the CNN model with the S3 image URL.
      - Invoke the CNN model to classify the image and get a prediction.

   g. If no hazard is predicted by FL/RFC, set CNN prediction to "No prediction available".

4. Update the DynamoDB record with:
   - FL/RFC prediction.
   - CNN prediction.
   - Image URL.

5. Return the predictions and image URL to the API (if invoked directly):
   - Send a response containing the predictions and image URL.

6. If an error occurs at any point:
   - Log the error.
   - Return an error response with a 500 status code.
```

Figure 14: ProcessSensorData Lambda Function pseudocode

In comparison to the python script used to handle the edge layer, the lambda function is written in JavaScript. The lambda function code begins by initializing all the necessary AWS runtime clients: S3, IoT, DynamoDB, and SageMaker. These are used to access signed image URLs, access the database, and to invoke SageMaker endpoints that store the trained models for sensor and image analysis. As part of these runtime clients, supporting variables and constants are defined to access these services.

Once setup is complete, the lambda goes through each record in the DynamoDB stream indicating a new item has been inserted. For this model only one record containing all the sensor data is inserted per iteration. Therefore, the lambda retrieves the inserted record and prepares the data to be sent into the first SageMaker endpoint. This endpoint holds the model for the Fuzzy Logic and Random Forest Classifier hybrid. Figure 15 shows the pseudocode of how the model is trained.

As reference, the SageMaker endpoints were created by creating custom built docker containers that are built on all the necessary libraries and tools required for the machine learning models to work properly. By using podman and docker, the container images were created and uploaded into AWS Elastic Container Registry (ECR). A custom-built repository in ECR stores the container image and works in coordination with AWS SageMaker to pull and create the machine learning model, endpoint configuration, and endpoint. This deployment process involves creating a compressed folder that includes a Docker file, requirements text file with corresponding dependencies, inference script (similar to training script but uses the ML trained model and uses API routes through flask to handle requests), and the trained machine learning model. This compressed folder is what's used to create the container images for both FL/RFC and CNN model endpoints.

```

1. Import Libraries
    Import necessary libraries: Pandas, NumPy, scikit-fuzzy, sklearn, joblib, boto3, etc.

2. Define S3 Bucket and File Path Information
    - Set up bucket name, folder paths, file names for sensor data, results, and models.

3. Define Fuzzy Logic and Utility Functions
    (apply_fuzzy_logic):
    - Create fuzzy variables for temperature, humidity, smoke, flame, sunlight, gas.
    - Define fuzzy memberships (e.g., low, moderate, high).
    - Define fuzzy output variable 'fire_detected' with states: 'no_fire', 'risk', 'fire'.
    - Define rules for detecting fire risk based on combinations of inputs.
    - Use fuzzy control system to compute fire risk for given sensor data.
    (prepare_data): preprocess and prepare features and labels
    (map_decision_to_string): maps numeric values back to string labels after predictions
    (train_model): RFC train function
    (load_data_from_s3): loads data from s3 like csv file
    (save_model_to_s3): uploads trained model to s3 bucket
    (upload_to_s3): upload logic to s3 bucket

4. Main Logic
    - Load Data from S3:
      - Connect to S3 bucket.
      - Download CSV file containing sensor data.
      - Load data into a Pandas DataFrame.
    - Apply Fuzzy Logic:
      - Iterate through DataFrame rows.
      - Apply 'apply_fuzzy_logic' function to calculate 'fire_detected' values.
      - Add results as a new column in the DataFrame.

5. Prepare Data for Machine Learning Model
    - Define input features (e.g., temperature, humidity, flame, etc.).
    - Define target variable ('fire_detected').
    - Split data into training and testing sets.

6. Train Random Forest Classifier (RFC)
    - Initialize RFC model with parameters.
    - Train the model using the training set.
    - Evaluate the model on the test set:
      - Calculate accuracy.
      - Generate classification report.

7. Save Model and Results
    - Save the trained RFC model using 'joblib'.
    - Save results (e.g., accuracy, classification report) to a text file.

8. Upload Updated Files to S3
    - Upload updated CSV file with 'fire_detected' values back to S3.
    - Upload the trained RFC model and results text file to S3.

9. Handle Errors
    - Use try-except blocks to catch errors (e.g., missing AWS credentials).

10. Output
    - Print accuracy and classification report to console.
    - Confirm successful uploads to S3.

```

Figure 15: FL/RFC Hybrid Model Pseudocode

The FL/RFC hybrid model is written in python and begins with basic library imports required to perform connections and operations. Along with the CNN model which will be discussed later in this section, both models were trained using open-source datasets [21-31] that contain data for both outdoor and indoor scenarios. For the FL/RFC model, a total of 5000 records were used as part of the training process. The FL/RFC model also contains the defined folder paths, file names, and variables to access necessary resources. Next, a set of functions are defined to apply fuzzy logic as well as prepare data, access S3, and train the hybrid model. Within the “apply_fuzzy_logic” function a couple of operations are performed. First, the fuzzy variables for each of the different sensors are created. This is followed by the definition of all the fuzzy memberships for each sensor. Table 1 shows the defined fuzzy membership for temperature, while Figure 16 illustrates the membership function plot. All fuzzy memberships and plots were created using MATLAB as a tool to create a Sugeno Fuzzy logic system.

Membership	Range (°F)
cold	$-67 \leq X \leq 67$
moderate	$60 \leq X \leq 90$
hot	$82 \leq X \leq 112$
extreme	$137 \leq X \leq 257$

Table 1: Temperature Fuzzy Membership

The fuzzy membership for temperature is divided into four categories representing: cold, moderate, hot, and extreme. Most memberships were based off the ranges in [7] including

temperature, however minimum adjustments were made across membership definitions to create an overlap of 50% between each membership. This adjustment was done in the context of balancing coverage and efficiency for uncertain values reflecting real world variability.

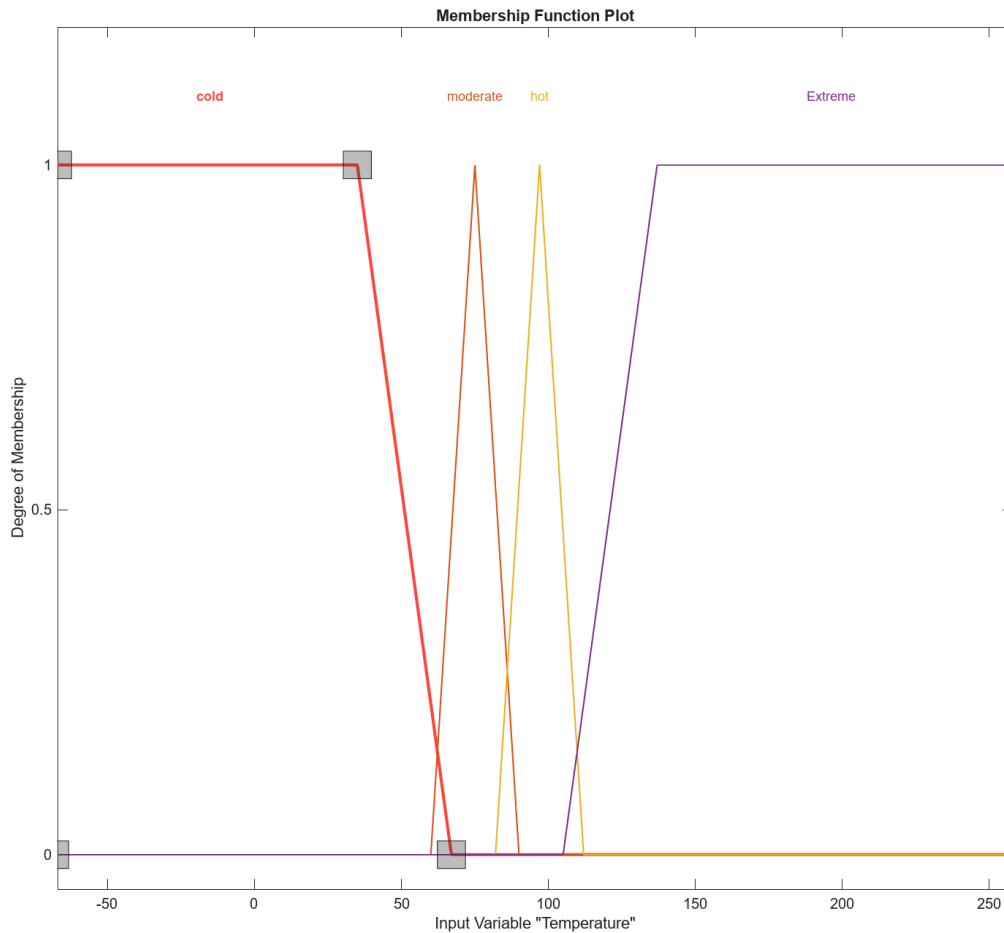


Figure 16: Temperature Membership Function Plot

As shown in Figure 16, each membership overlaps each other to accomplish the 50% overlap strategy. In this set of particular memberships both ‘cold’ and ‘extreme’ are defined as trapezoidal shapes to help define smooth transitions between the ‘moderate’ and ‘hot’ triangular membership shapes, which is crucial for the fuzzy inference system.

Table 2 shows the fuzzy memberships for smoke: low, moderate, and high. They are categorized based off the intensity of smoke concentration. In this definition, ‘low’ and ‘high’ are defined as a trapezoidal shape, while ‘moderate’ reflects a triangular shape. Figure 17 illustrates this degree of belonging between the membership functions.

Membership	Range (ppm)
low	$0 \leq X \leq 65$
moderate	$43 \leq X \leq 152$
high	$126 \leq X \leq 1000$

Table 2: Smoke Fuzzy Membership

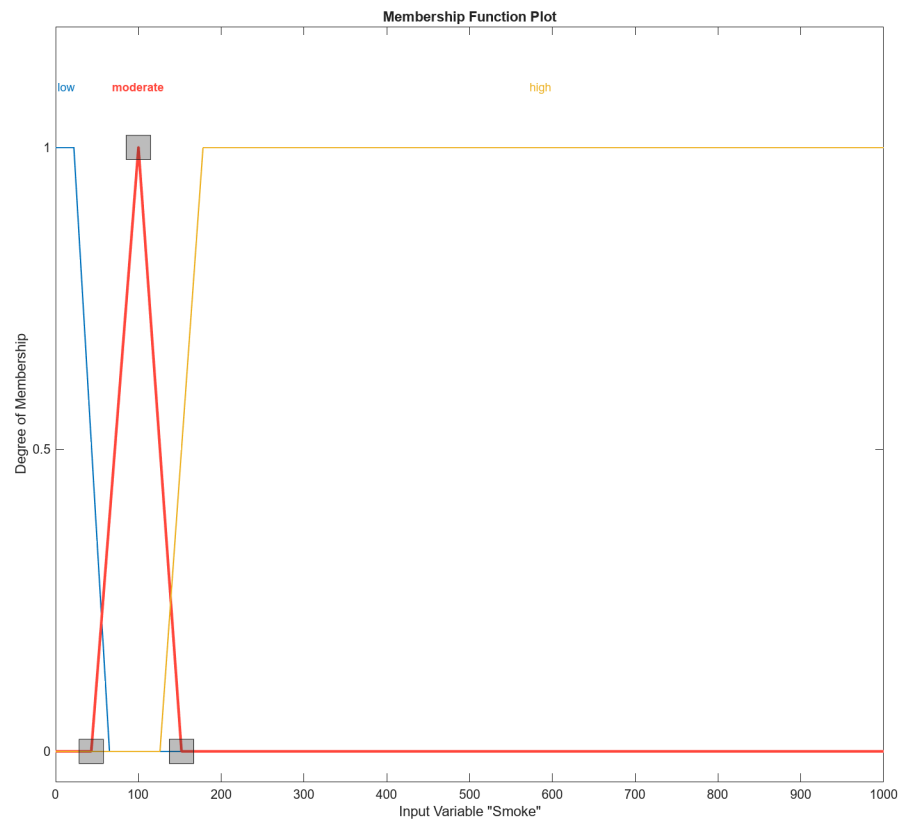


Figure 17: Smoke Membership Function Plot

Table 3 shows the flame fuzzy membership. This particular membership is unique in the sense that a binary logic approach is set. As a result, 0 represents no fire and 1 represents the presence of fire. In Figure 18, this relationship can be observed. Due to the nature of the binary membership function, the memberships differ to other traditional fuzzy logic memberships where there is no overlap, reflecting simplicity.

Membership	Range (binary)
no	$X = 0$
yes	$X = 1$

Table 3: Flame Fuzzy Membership

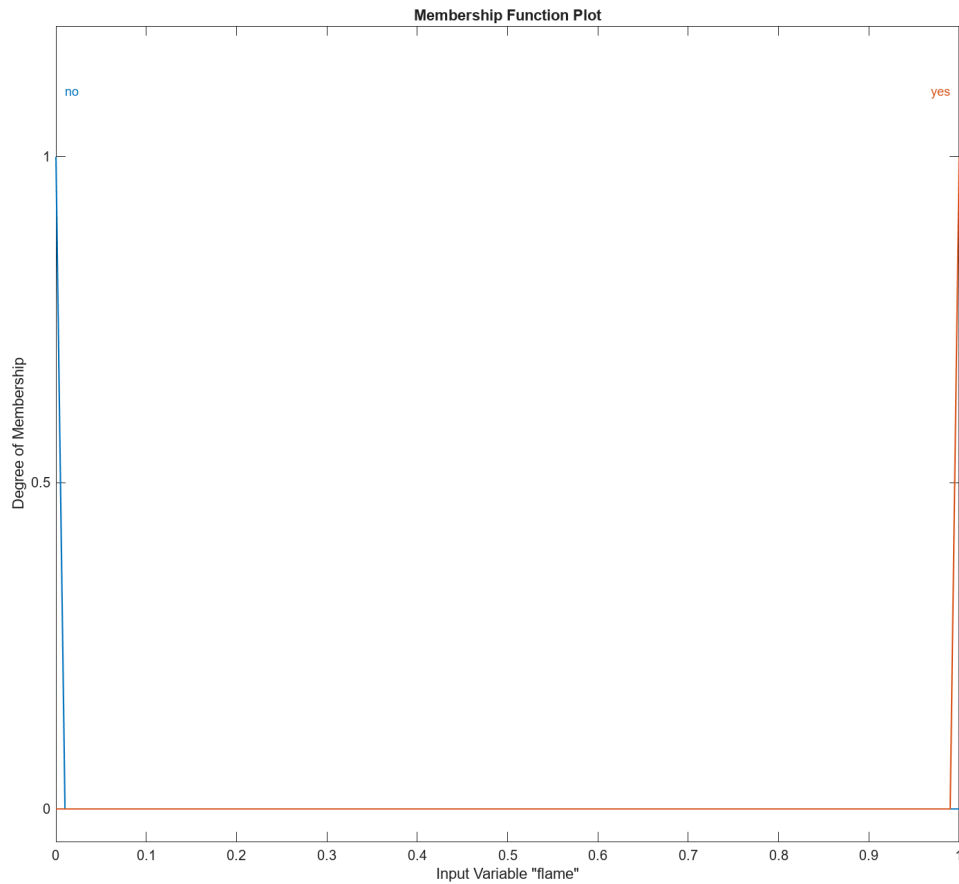


Figure 18: Flame Membership Function Plot

In Table 4 the gas fuzzy memberships are categorized into three memberships: low, moderate, and high. Each one represents a range of gas concentrations. For this fuzzy set, the ‘low’ and ‘high’ memberships are trapezoidal shaped, while the ‘moderate’ membership is triangular. As with the other sensor memberships, Figure 19 illustrates the overlapping membership plot for gas.

Membership	Range (ppm)
low	$0 \leq X \leq 95$
moderate	$76 \leq X \leq 152$
high	$133 \leq X \leq 1000$

Table 4: Gas Fuzzy Membership

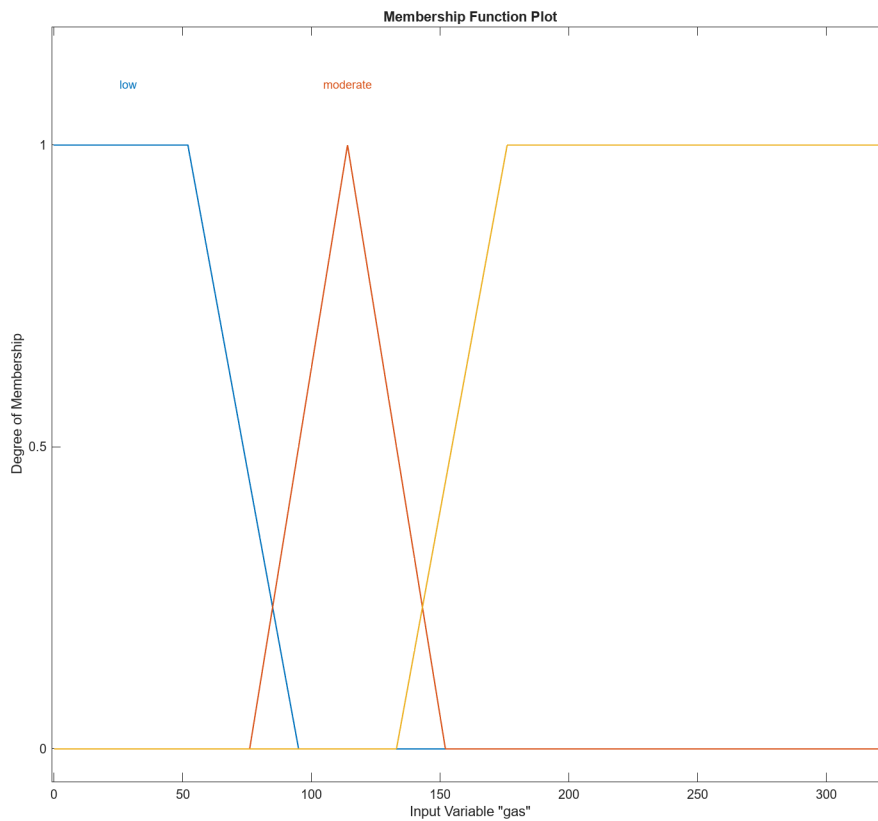


Figure 19: Gas Membership Function Plot

The sunlight fuzzy memberships are shown in Table 5. The ranges relate to the intensity of ambient light conditions. This membership was categorized into four conditions depicting: indoor, overcast, strong, and direct. A trapezoidal shape was given to ‘indoor’ and ‘direct’ while the ‘overcast’ and ‘strong’ memberships were given a triangular shape. These can be observed in Figure 20 as they overlap each other.

Membership	Range (lux)
indoor	$0 \leq X \leq 10000$
overcast	$7500 \leq X \leq 22500$
strong	$20000 \leq X \leq 32500$
direct	$30000 \leq X \leq 100000$

Table 5: Sunlight Fuzzy Membership

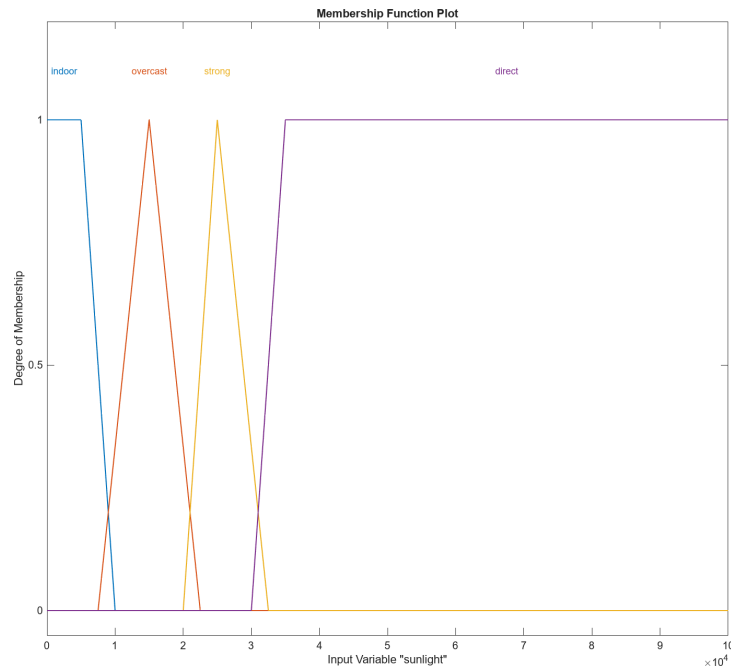


Figure 20: Sunlight Membership Function Plot

Membership	Range (%)
low	$0 \leq X \leq 42$
moderate	$34 \leq X \leq 70$
high	$61 \leq X \leq 100$

Table 6: Humidity Fuzzy Membership

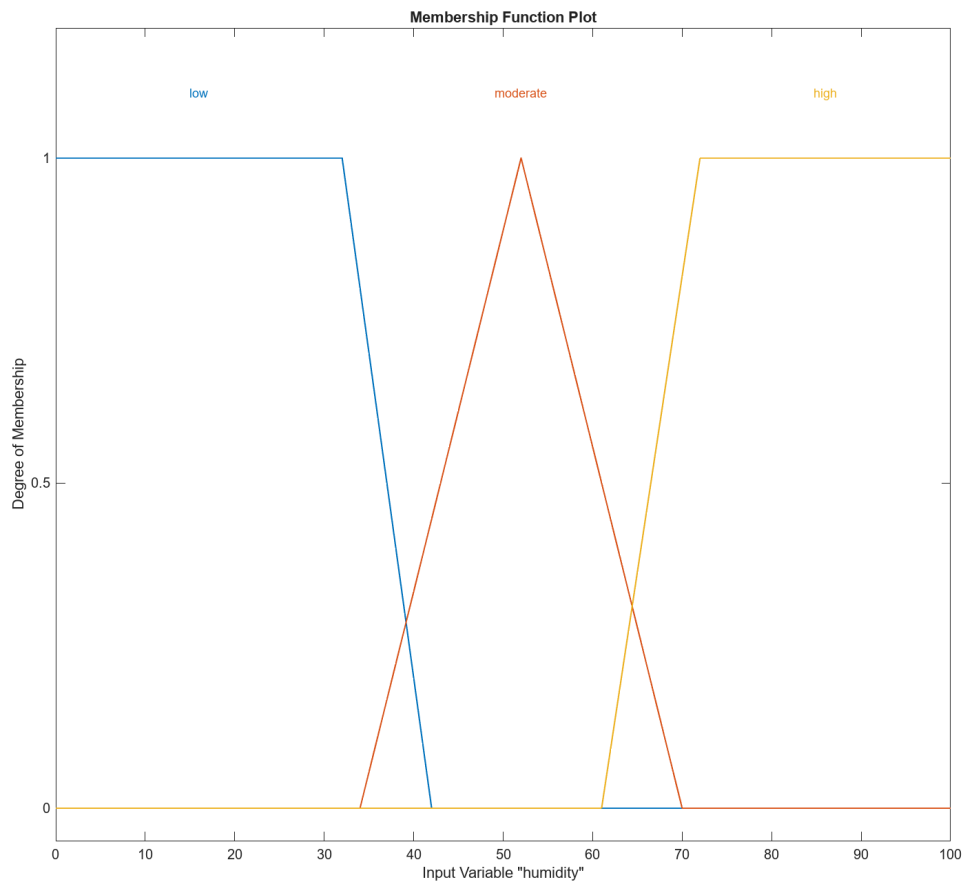


Figure 21: Humidity Membership Function Plot

The last membership as part of the input variables was for a humidity fuzzy membership.

Table 6 shows the categories given to this fuzzy set: low, moderate, and high. Similar to the

previous membership declarations, the humidity membership for ‘low’ and ‘high’ were given trapezoidal shapes. The ‘moderate’ membership was given a triangular shape.

To expand further, trapezoidal and triangular shapes were chosen due to the ease of interpretation and smooth transition between adjacent fuzzy sets. In extreme ends such as the beginning and end of memberships, trapezoidal shapes help handle the boundaries by accommodating smooth transitions that may cause uncertainty. On the other hand, using triangular shapes in middle range values allows clear overlapping transitions with sharp peaks that represent more confident regions of the membership. By combining both shapes, a more dynamic approach for real world instances can be stabilized for improved accuracy.

Finally, a fuzzy membership was also declared for the output of the fuzzy logic system, ‘fire_detected’. The output for this model declares ‘no_fire’, ‘risk’, and ‘fire’ memberships. In this fuzzy membership, the decision-making outputs related to a potential fire are shown within Table 7. The ranges are part of the defuzzification process that computes crisp values indicating states of the system.

Membership	Range
no_fire	$0 \leq X \leq 35$
risk	$15 \leq X \leq 90$
fire	$70 \leq X \leq 100$

Table 7: Fire Detected Fuzzy Membership

Figure 22 shows the illustration of the ‘fire_detected’ fuzzy membership function plot.

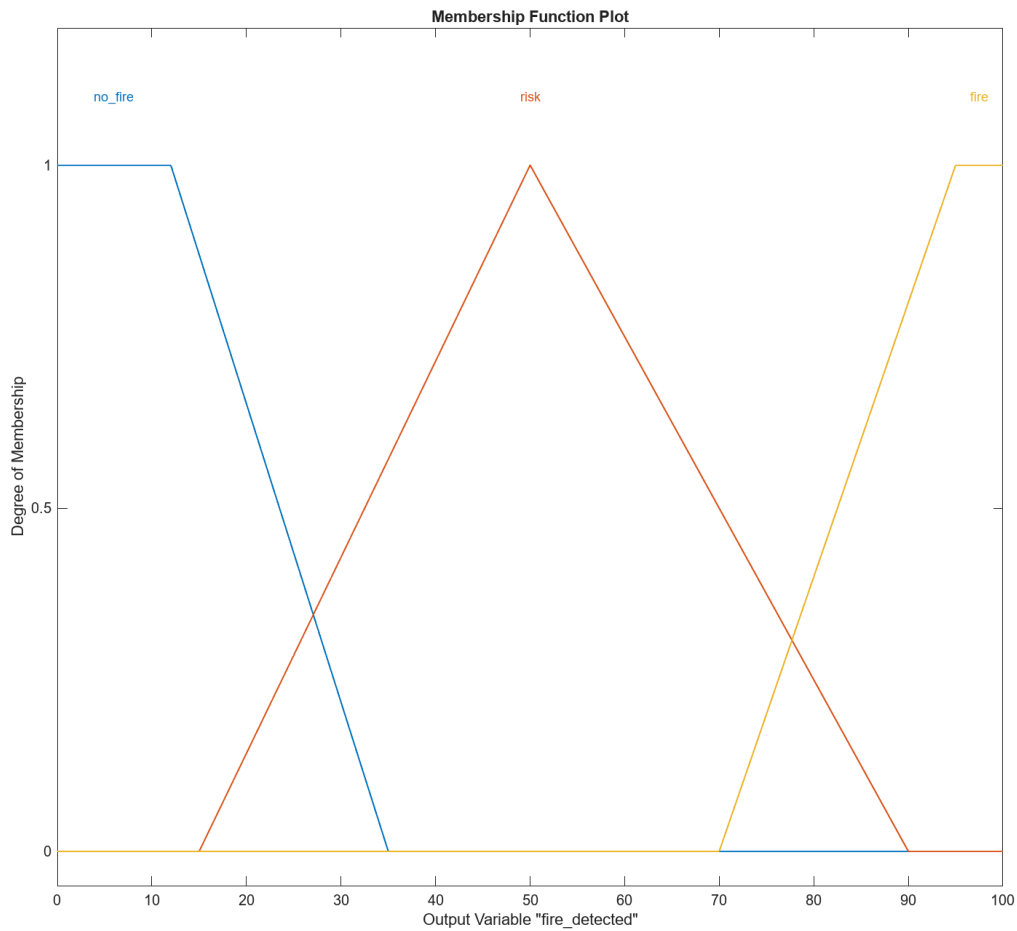


Figure 22: Fire Detected Membership Function Plot

The plot enables a visual of the degree of membership between ‘no_fire’, ‘risk’, and ‘fire’, where a trapezoidal shape represents the ‘no_fire’ and ‘fire’ states. As the middle range of this membership, the ‘risk’ state is given a triangular shape.

Referring back to the pseudocode in Figure 15, after defining the membership functions and variables, the rules make up the last part of the fuzzy logic system. The rules were generated with the help of MATLAB and were simplified through the use of IF, OR, THEN, ELSE

statements. Table 8 shows the different combinations that could be defined based off the combination of inputs and their degree of membership in the fuzzy logic system, it is color coded to help see the threat degrees of each input. The output column serves as the final decision of what the fire-based risk is determined, enabling it to handle uncertainty and variability effectively.

Temperature	Smoke	Flame	Gas	Humidity	Output
Cold/Moderate	Low/Moderate	No	Low/Moderate	Low/Moderate/High	No Fire
Cold/Moderate	Low/Moderate	No	High	Low/Moderate	Risk
Cold/Moderate	Low/Moderate	No	High	High	No Fire
Cold/Moderate	Low/Moderate	Yes	Low	Low/Moderate/High	No Fire
Cold/Moderate	Low/Moderate	Yes	Moderate/High	Low/Moderate	Fire
Cold/Moderate	Low/Moderate	Yes	Moderate/High	High	Risk
Cold/Moderate	Moderate/High	No	Low	Low/Moderate	Risk
Cold/Moderate	Moderate/High	No	Low	High	No Fire
Cold/Moderate	Moderate/High	No	Moderate/High	Low/Moderate	Fire
Cold/Moderate	Moderate/High	No	Moderate/High	High	Risk
Cold/Moderate	Moderate/High	Yes	Low	Low/Moderate	Fire
Cold/Moderate	Moderate/High	Yes	Low	High	Risk
Cold/Moderate	Moderate/High	Yes	Moderate/High	Low/Moderate	Fire
Cold/Moderate	Moderate/High	Yes	Moderate/High	High	Risk
Hot/Extreme	Low	No	Low	Low/Moderate	No Fire
Hot/Extreme	Low	No	Low	High	No Fire
Hot/Extreme	Low	No	Moderate/High	Low/Moderate	Risk
Hot/Extreme	Low	No	Moderate/High	High	Risk
Hot/Extreme	Moderate/High	No	Low	Low/Moderate	Risk
Hot/Extreme	Moderate/High	No	Low	High	Risk
Hot/Extreme	Moderate/High	No	Moderate/High	Low/Moderate	Fire
Hot/Extreme	Moderate/High	No	Moderate/High	High	Risk
Hot/Extreme	Low	Yes	Low	Low/Moderate	Risk
Hot/Extreme	Low	Yes	Low	High	Risk
Hot/Extreme	Low	Yes	Moderate/High	Low/Moderate	Fire
Hot/Extreme	Low	Yes	Moderate/High	High	Risk
Hot/Extreme	Moderate/High	Yes	Low	Low/Moderate	Fire
Hot/Extreme	Moderate/High	Yes	Low	High	Risk
Hot/Extreme	Moderate/High	Yes	Moderate/High	Low/Moderate	Fire
Hot/Extreme	Moderate/High	Yes	Moderate/High	High	Fire

Table 8: Fuzzy Logic Rules

After computing the risk of the given sensor data through the fuzzy control system, the hybrid FL/RFC model prepares the features taken from fuzzy logic through the use of pandas Data

frames. These include: temperature, smoke, flame, gas, humidity, and sunlight. In addition, it also prepares the labels, or output of the fuzzy logic system, which is the 'fire_detected' label. These values are mapped from string labels to numeric values so that it satisfies the data types required by the RFC model for training. The purpose of using the fuzzy logic system along with the RFC is due to the following reasons:

- Raw sensor data may be noisy or have subtle variations that make classification less reliable without sufficient preprocessing. By using FL prior to RFC, it converts the already simplified inputs to improve the RFC's ability to identify patterns.
- By using FL, it remediates the complex decision boundaries that RFC alone would face when learning relationship between raw sensor data readings and output classification. Thus, decreasing the overall training complexity.
- By using FL, inputs are translated into higher level descriptors that are more stable and meaningful than raw readings in terms of uncertainty through the use of the degree of memberships, which tend to lead to misclassification of data.
- Through the use of FL, a layer of interpretability is derived to help understand how raw inputs are transformed, which makes it easier to understand.

Figure 23 shows a perspective of the trained FL/RFC model. The RFC takes the features and labels from the FL system and makes node decisions through tree splitting. A basic example of how the tree traverses through each node can be represented by a statement such as: IF temperature is hot > 100 AND humidity is dry < 20 THEN go left, else go right.

This makes reference to how the tree traverses through the different features to determine what decision is taken. In this specific model, instead of using threshold values like the given example,

a fuzzy logic value is used within each node to traverse depending on the degree of memberships.

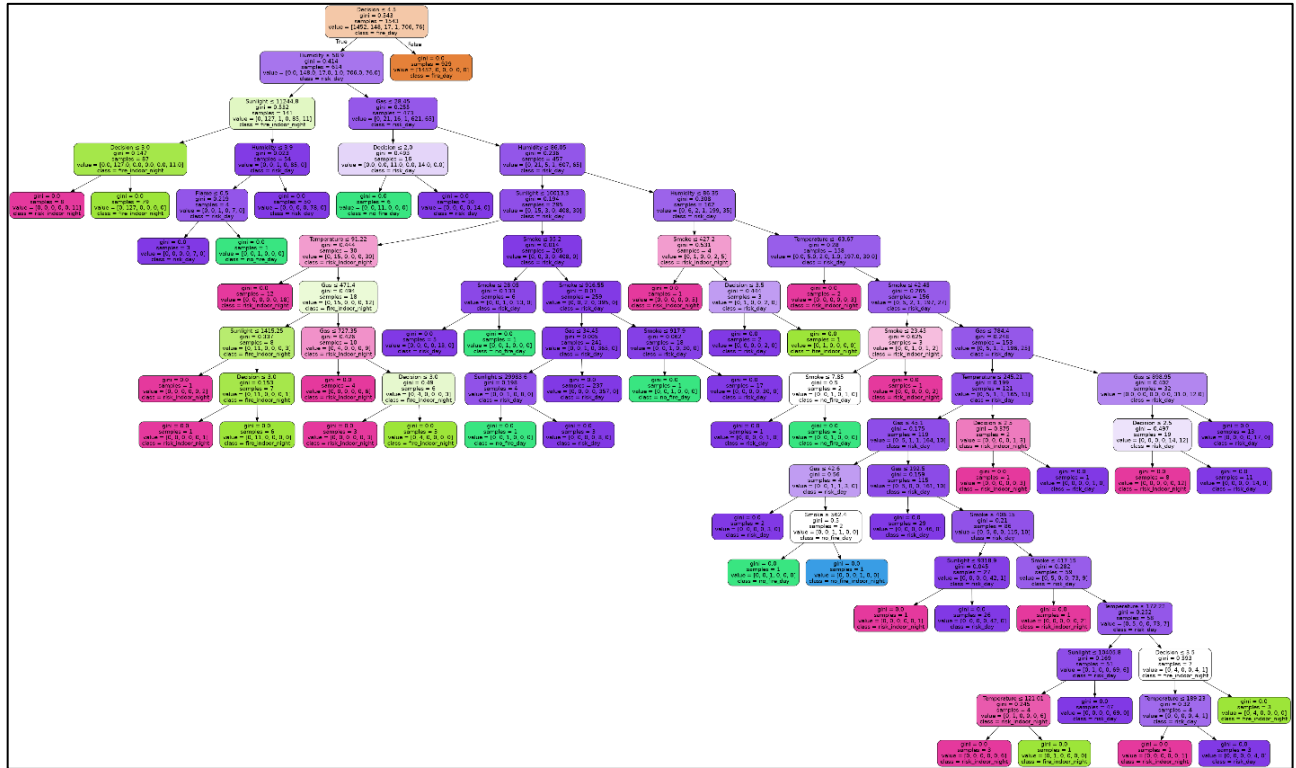


Figure 23: Random Forest Classifier Visual

Once the model has been trained the script loads csv data with decisions for every row of data that was used for training from the datasets. The script also handles uploading the trained model into S3 along with a text file containing all the performance metrics. This model is what is used in the AWS SageMaker endpoint that is invoked in the processSensorData lambda function.

Returning to discussion of the process going on in Figure 14 for the lambda function. When the lambda invokes the FL/RFC model, a decision is returned: no_fire_indoor_night, no_fire_day, risk_indoor_night, risk_day, fire_indoor_night, and fire_day. This decision is used to determine whether the event is a potential fire threat. In cases where the endpoint returns a risk or fire prediction across the indoor, outdoor, or night scenarios, then a MQTT message that

communicate from the cloud layer to the edge layer signals the Raspberry Pi to capture and store an image into AWS S3. The lambda then prepares this image and invokes the second AWS SageMaker endpoint that stores the CNN model, which does image processing.

```

1. Import required libraries: TensorFlow, boto3, os, ImageDataGenerator, matplotlib, numpy, and sklearn.metrics.

2. Define S3 configuration:
   - Set bucket name, image folder path, local image directory, model folder path, and result text file path.

3. Initialize S3 client.

4. Define function `download_images_from_s3`:
   - Create local directory if it doesn't exist.
   - Use a paginator to iterate through objects in the S3 bucket.
   - Download images from S3 to the local directory.

5. Define function `create_model`:
   - Load ResNet50 as the base model with pre-trained weights.
   - Unfreeze the last 10 layers for fine-tuning.
   - Build a Sequential model with:
     - GlobalAveragePooling2D layer.
     - Dense layer with L2 regularization and ReLU activation.
     - BatchNormalization layer.
     - Dropout layer.
     - Dense layer with softmax activation for multi-class classification.
   - Compile the model with Adam optimizer, categorical cross-entropy loss, and accuracy metrics.

6. Define function `plot_history`:
   - Plot training and validation accuracy.
   - Save the plot locally and upload it to S3.

7. Define function `preprocess_image`:
   - Load and resize an image.
   - Convert the image to an array and normalize its values.

8. Define function `predict_image`:
   - Predict probabilities for each class using the model.
   - Identify the predicted class label and its confidence score.

9. Define function `save_model_to_s3`:
   - Save the model in `.h5` format locally.
   - Upload the saved model to S3.

10. Define `main` function:
    - Step 1: Download images from S3.
    - Step 2: Initialize `ImageDataGenerator` with augmentation and validation split.
    - Step 3: Load training and validation data using `flow_from_directory`.
    - Step 4: Create the model using `create_model`.
    - Step 5: Define callbacks for early stopping and learning rate reduction.
    - Step 6: Train the model using `model.fit` with specified epochs, training data, validation data, and callbacks.
    - Step 7: Plot training history and upload the plot to S3.
    - Step 8: Evaluate the model on validation data and print accuracy.
    - Step 9: Generate and print a classification report using validation data.
    - Step 10: Save the trained model to S3.
    - Step 11: Save validation results and classification report to a text file and upload it to S3.
    - Step 12: Print confirmation of model training completion.

11. If script is executed, call the `main` function.

```

Figure 24: CNN Model Pseudocode

The CNN model is written in python, the script starts by importing all the required libraries to perform image analysis and calculations. This is followed by defining the S3 bucket configuration that contains all the images as part of the training dataset. A total of 4000 images were used as part of this process. The script also contains a helper function that assists in downloading these images for training. Next, the script defines the model that is created to handle all image processing within a function called 'create_model'. This model is based off the ResNet50 architecture [32] which is a pretrained model that can be fine-tuned for specific tasks to save time and improve accuracy. In the implementation for this thesis, the ResNet50 architecture is loaded with pretrained weights and set to process images with a 296 x 296 size. The script handles fine tuning by unfreezing some layers of the pretrained model to train based off the image dataset of fire scenarios, which allowed the model to adapt to the specific features we needed. The function also included global average pooling to help convert the feature map by averaging feature values across the dimensions. This helped reduce parameters to reduce overfitting. The model also included dense layers, which fully connected the layers to learn class specific patterns through ReLU activation and L2 regularization. The addition of batch normalization also helped stabilize training and improve convergence speed. A dropout layer was also part of the model creation to drop layers during training, which serves as a robust feature that forces the model to drop already learned features to prevent it from relying on them. Finally, a dense layer with SoftMax activation allowed the model to have multi-class classification for the four classifications used in this work. The model is then compiled with Adam optimizer for efficient training and adaptive learning rate, as well as metrics to differentiate between predicted class probabilities. Ultimately, most of these techniques along

with data augmentation were used to prevent overfitting and to help enhance the metrics of the model. Figure 25 shows the layers of the CNN model as described.

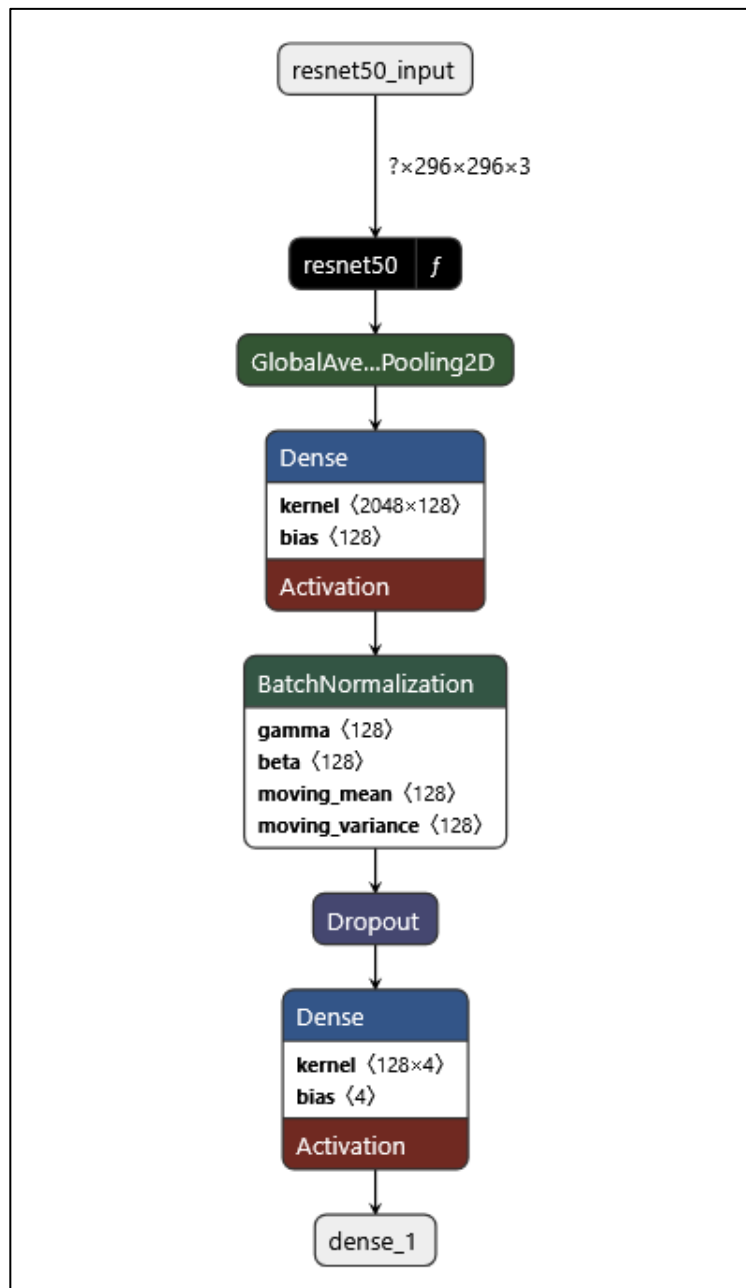


Figure 25: ResNet50 based Architecture for CNN Model

In this study a built from scratch model was developed as well to try and evaluate performance metrics. Figure 26 shows this model.

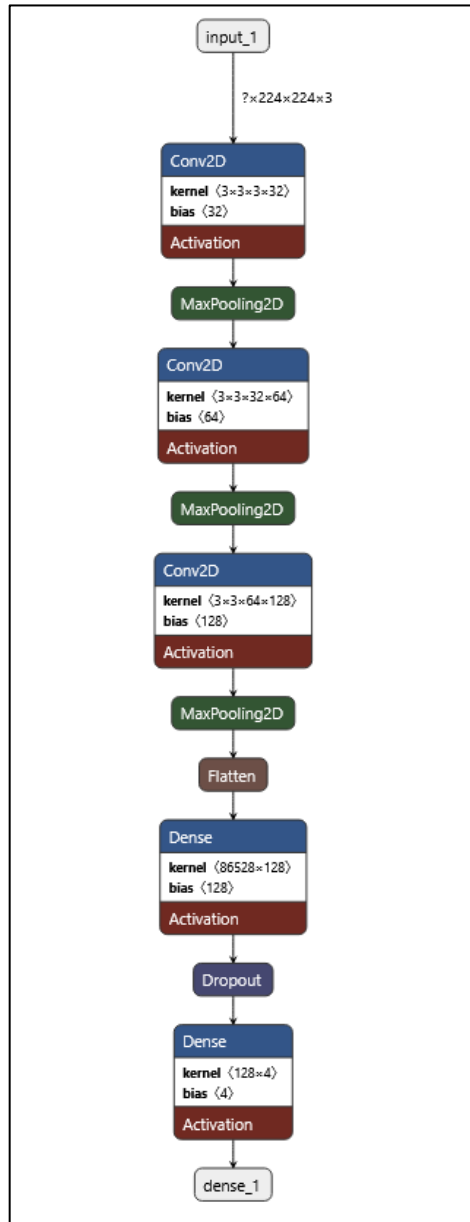


Figure 26: Built from scratch CNN Model

The model attempted to build from scratch lacked certain approaches in comparison to the ResNet50 approach. For example, batch normalization and some data augmentation techniques used in the ResNet50 architecture. However, it served to gain an understanding of the differences in using pretrained models against building a model from scratch.

After the model function definition, the training script had a function to plot the performance metrics such as validation accuracy of the model for when training was completed. This plot was uploaded to S3 for documentation. The script also consisted of a pre-process and predict image function, which helped load and resize the images as well as predicting the probabilities of each class using the model. There was also logic to save and upload the trained model to S3, which was part of creating the endpoint. Finally, the main function of the script contained all the function calls of the functions created to pre-process and train the model based off the downloaded images from S3 and generate the report of all metrics, then save them for record keeping.

In continuation of the lambda function pseudocode, it invokes the trained CNN model recently discussed to get back a final decision. Once the predictions from both SageMaker endpoints are received, the lambda updates the recently submitted sensor data with its respective decisions and the image URL of the captured Raspberry Pi picture. A 200-response code is returned indicating a successful lambda execution, or a 500-status code with an error message indicating the lambda failed at some point in execution. Error handling is implemented across the lambda function to ensure it continues functionality, as this is a very crucial architecture component of the proposed fire detection model since major decision making occurs at this point. One of the features within the lambda is that it does not require the CNN endpoint to

return a prediction. For example, if the FL/RFC model returns a decision that indicates no fire is present, it skips invoking the CNN endpoint prediction.

3.4 UI Layer Web based Application Dashboard

The last section of the methodology involves the UI layer. In this layer, a web-based dashboard is created to display the status of the fire detection model in real-time. It is based on a Next.js framework and basic CSS styling. The basic functions within the dashboard involve data fetching, rendering, user interaction, and AWS integration.

High level architecture of the dashboard involves dynamic data fetching, state management, and API routes. The user interface is built using react components for reusability, which includes features like the auto refresh toggling button, error handling, and styling across the tables. As part of the API routes, a GET method fetches sensor data from the DynamoDB table through a scan using the table name as parameters. This is dynamically possible due to the useEffect react hook. Other applications of react components throughout the dashboard include: displaying features like image URL, error statuses, sensor data, and the refreshing mechanism of the page that allows the user to refresh upon request or let it refresh automatically every couple of seconds. When the refresh occurs, all fetched sensor data is immediately available to view in the dashboard table along with the predictions and the image. Figure 27 shows a visual of the dashboard and its features.

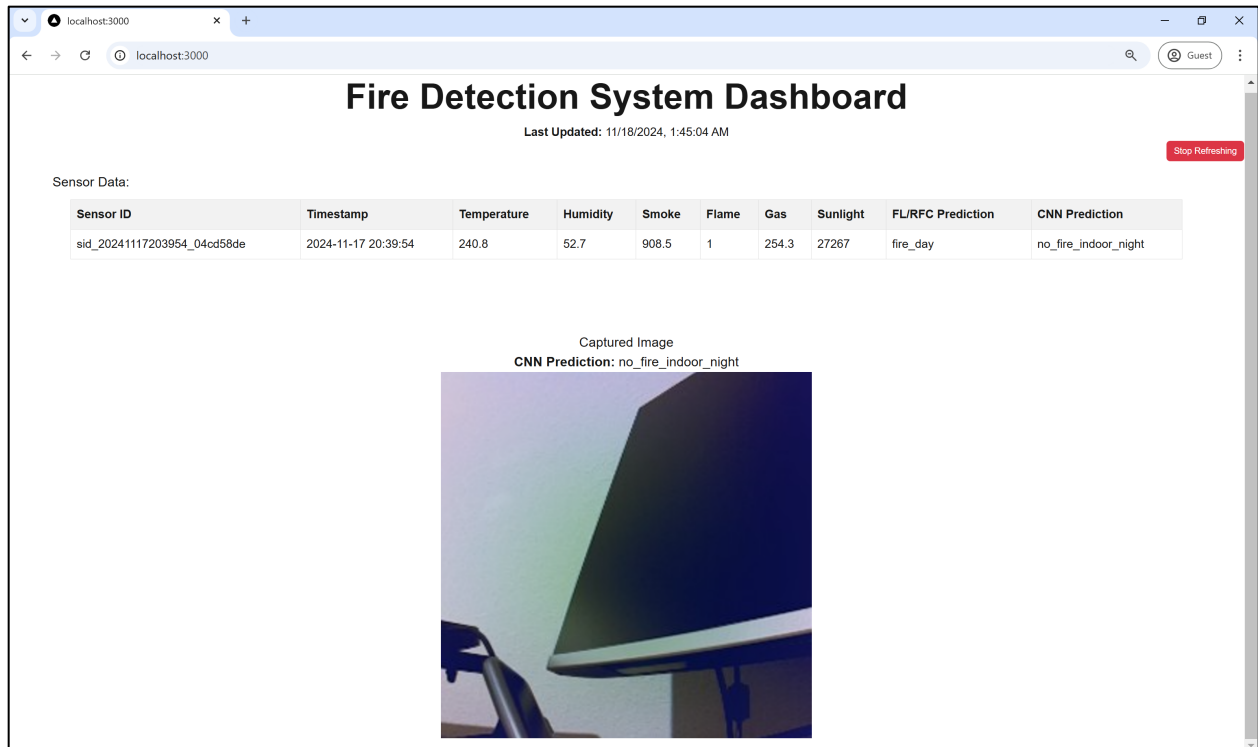


Figure 27: Web Application Dashboard Interface

The dashboard was also built to handle errors such as when any sensor data, images, or predictions are not available at the time of refreshing and fetching. A detail included on the dashboard was to display the time of when it was last updated to have an understanding of how it matches against the timestamp of the last retrieved sensor information. This provides insight towards real-time values and if the dashboard is up to date.

CHAPTER 4: RESULTS

This chapter discusses some of the results obtained when testing the hybrid model against performance metrics like accuracy, response time, precision, recall, and F1 scores. It also briefly discusses the approach taken to obtain these results.

The first part of the model tested was the FL/RFC model, which used historical data from open-source datasets as mentioned earlier in the thesis. This model was trained using approximately 5000 records containing different sensor readings ranging from outdoor and indoor scenarios. As part of the training this data was split into 80% for training and 20% for validation accuracy. A JupyterLab notebook within AWS SageMaker was configured to run the training script without having to worry about hardware limitations. Figure 28 shows the performance metrics obtained after completing the training.

Accuracy: 0.999				
Classification Report:				
	precision	recall	f1-score	support
fire_day	1.00	1.00	1.00	452
fire_indoor_night	1.00	1.00	1.00	56
no_fire_day	1.00	1.00	1.00	176
no_fire_indoor_night	1.00	0.95	0.98	22
risk_day	1.00	1.00	1.00	252
risk_indoor_night	0.98	1.00	0.99	42
accuracy			1.00	1000
macro avg	1.00	0.99	0.99	1000
weighted avg	1.00	1.00	1.00	1000

Figure 28: FL/RFC Results

As seen, the training results provided a 99.9% accuracy. Some slight performance metrics that were affected were related to indoor and night domains. This was mainly due to a shortage of data in comparison to the other classification options. Regardless, the FL/RFC combination produced high performance metrics overall. These results were very similar to the ones obtained by the authors in [8] where they obtained 100% accuracy.

For the CNN model implemented in this thesis a total of 4000 images were gathered from distinct open-source datasets. The training followed the exact process as the one used for the FL/RFC model where a JupyterLab notebook was configured within AWS SageMaker to avoid hardware restrictions, especially since image processing is more aggressive on resources. The data again was split 80% for training and 20% for validation accuracy. Figure 29 shows the performance metrics obtained by the CNN model using the ResNet50 architecture. A total of 88.47% accuracy was obtained, however in this model there were larger effects on the precision, recall, and f1-score across all the categories. It is believed that similar features across the different features played a role in these results. Many of the open-source images had very similar backgrounds and fire textures for fire-based datasets, the same was seen across data not containing fire.

Validation Accuracy: 88.47%					
Classification Report:					
		precision	recall	f1-score	support
	fire_day	0.30	0.29	0.29	200
fire	indoor_night	0.25	0.25	0.25	200
	no_fire_day	0.29	0.24	0.26	199
no_fire	indoor_night	0.29	0.35	0.32	199
	accuracy			0.28	798
	macro avg	0.28	0.28	0.28	798
	weighted avg	0.28	0.28	0.28	798

Figure 29: ResNet50 CNN Model Results

Figure 30 shows graph comparison between the training accuracy over the validation accuracy. As seen on the figure, both accuracies were very close to each other across the multiple epochs. This demonstrates that the model performed well in terms of overfitting, where larger gaps between these two typically signal performance flaws like overfitting.

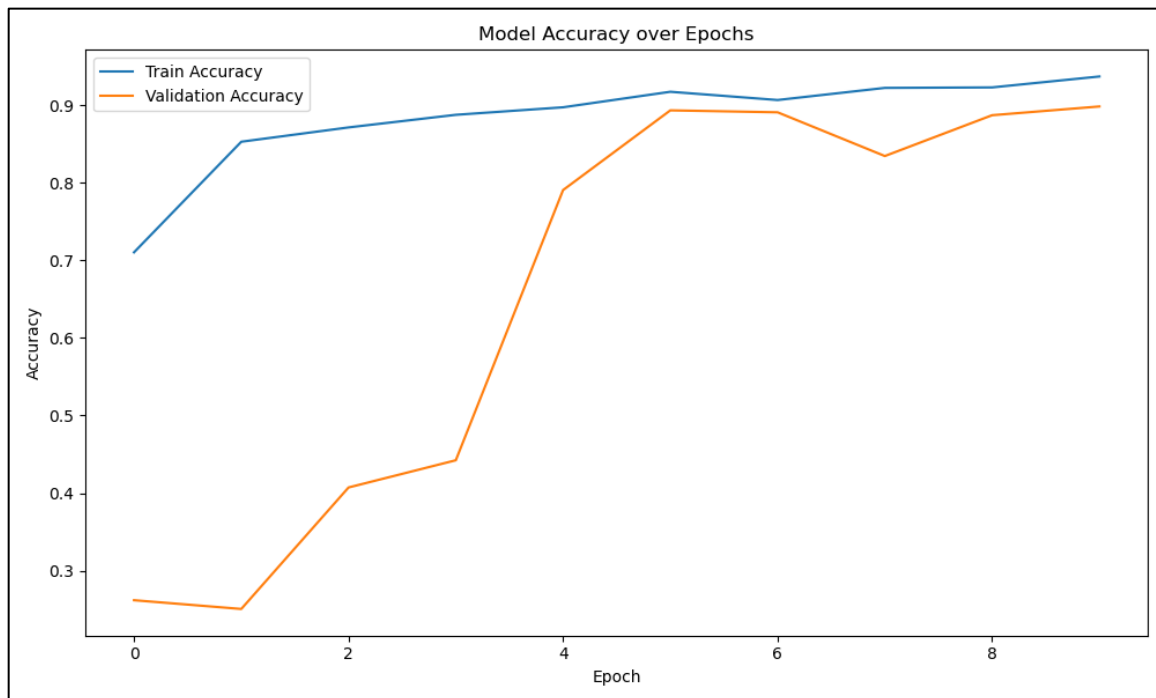


Figure 30: ResNet50 CNN Model Results Graph

In comparison to the ResNet50 based CNN model, the built from scratch model that was used as a foundation of this model obtained a lower accuracy of 68.50%. For this model the same images were used to compare how each model performed. As for the precision, recall, and f1-score, the results were poor similarly to the ResNet50 based CNN model. Since the same dataset was used it was concluded that a more diverse dataset with more diverse features for fire and non-fire

scenarios in indoors and outdoors domain could help improve these values. Figure 31 gives a detailed look at the performance metrics for the built from scratch model.

Validation accuracy: 68.50%					
Classification Report:					
		precision	recall	f1-score	support
	fire_day	0.25	0.37	0.30	100
fire	indoor_night	0.20	0.17	0.18	100
	no_fire_day	0.22	0.10	0.14	100
no_fire	indoor_night	0.30	0.35	0.32	100
	accuracy			0.25	400
	macro avg	0.24	0.25	0.23	400
	weighted avg	0.24	0.25	0.23	400

Figure 31: Built from Scratch CNN Model Results

Figure 32 shows the graph between the validation and validation accuracy for the built from scratch model. In comparison to the ResNet50 based model, the gap between both of these is larger, which means there were more hints of overfitting, especially since this model did not include techniques like batch normalization and data augmentation.

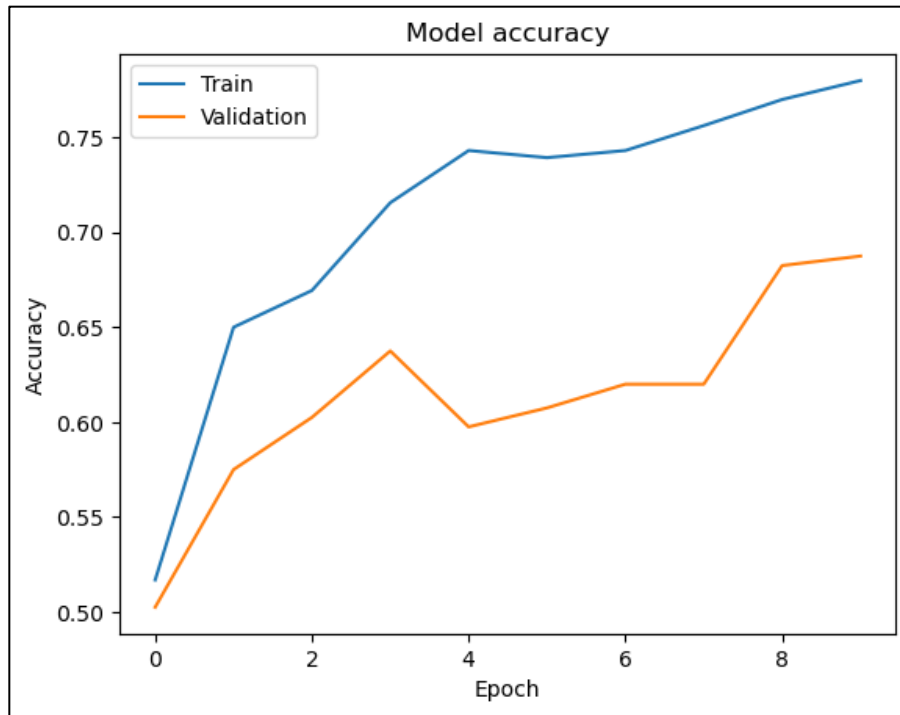


Figure 32: Built from Scratch CNN Model Results Graph

Ultimately, the machine learning models obtained good results, however, there are many approaches that could potentially improve the results obtained. It was also confirmed by these results that a pretrained model does indeed enhance feature extraction for specific scenarios like fire detection by making use of the improved feature extraction benefits they provide. In the creation of these models, the curve in the learning process also had an effect on the performance of these results as many of these were new areas of interest.

As part of the response time measurement of the hybrid's model ability to process real-time data a few logs were placed across the different layers of the architecture. These logs kept track of time as they reached certain part of the decision-making process. For example, these response times were measured:

- Response time from outlier detection to inserting into database (~500ms)

- From insertion to FL/RFC inference (~465ms)
- From insertion to CNN inference (~672ms)
- From insertion to end (~774ms)

By analyzing these measurements, the whole response time of the hybrid model pipeline had very responsive results. Data sent from sensors to dashboard was below 1s, which is a critical response time that benefits real-time detection of threats.

Finally, to determine if the hybrid model was able to handle false alarms, the use of artificial data was used to feed new data into the model. This data was based off the historical data gathered from the different datasets, especially in terms of ranges. Furthermore, the system was left running for a few hours in which fluctuation of data reflected similar metrics to unusual readings or sensor failures across the different sensors. After collecting an approximate total of 138 anomaly detections a query was ran on the database to determine what kind of predictions were detected by the FL/RFC model. Based off those ~138 records, no records were found containing no_fire scenarios in 'day' or indoor_night' domains. This information showed that the model was able to filter out data that did not portray any risks. By accomplishing this, resources are only used to handle potential threats and not segments of data that don't necessarily need further processing. Figure 33 and Figure 34 show the PartiQL queries used to retrieve and check the processed data. While it is true that using artificial data is probably not the best approach, it was the best available procedure to try and test this functionality.

Query 1

```
1 select * from sensorData where "flRFCPrediction" = 'fire_indoor_night' or "flRFCPrediction" = 'fire_day' or "flRFCPrediction" = 'risk_day' or "flRFCPrediction" = 'risk_indoor_night'
```

Run Clear

Table view JSON view

Completed
Started on 11/26/2024, 6:24:02 AM
Elapsed time 332ms

Items returned (138)

Download results to CSV

imageUrl	Flame	Smoke	timestamp	gps_coordinates	sensorid	Gas	flRFCPrediction	Temperature	Sunlight	Humidity	cnnPrediction
https://fdsdat...	0	547.2	2024-10-2...	(-39.676069, -92.2...	sid_202410...	235.7	fire_day	31.28	95692.8	5	no_fire_indoor_night
https://fdsdat...	0	875	2024-11-1...	(-86.402371, -133...	sid_202411...	181.1	risk_day	251.78	26056.6	77.6	no_fire_indoor_night
https://fdsdat...	1	128.7	2024-10-2...	(11.434262, -166...	sid_202410...	977.1	fire_day	174.92	20192.5	38.6	no_fire_indoor_night
https://fdsdat...	1	224.7	2024-11-1...	(-46.332028, 52.5...	sid_202411...	659.8	fire_day	-6.52	46013.6	59.1	no_fire_indoor_night

Figure 33: Sensor Data Fire or Risks Query

Query 1

```
1 select * from sensorData where "flRFCPrediction" = 'no_fire_indoor_night' or "flRFCPrediction" = 'no_fire_day'
```

Run Clear

Table view JSON view

Completed
Started on 11/26/2024, 6:22:38 AM
Elapsed time 95ms

The command has been executed successfully.

Items returned

Download results to CSV

No results
The query did not return any results.

Figure 34: Sensor Data No Fire or Risks Query

CHAPTER 5: DISCUSSION AND FUTURE WORK

This chapter opens discussion towards some of the challenges faced when working on this model. It also discusses some of the recommendations for future work and improvements.

5.1 Challenges

Developing a hybrid fire detection model presents unique challenges across the various stages. In beginning stages such as data gathering, searching for data relevant to the chosen sensors in the model can have limitations. Due to the nature of fire, not many datasets exist outside of forest fires. In particular, most indoor fire datasets involve data obtained from room-controlled fires. Having more options could definitely improve evaluation of models such as the proposed hybrid model.

In the planning stage, determining which algorithm to use for specific parts of the architecture can also be a major challenge. The field of machine learning continues to grow and with the variety of algorithms, an individual with limited background on these can result in a huge learning curve. An example, was trying to figure out the difference between a training and inference script and their differences. Also, in terms of how they are used in SageMaker to create endpoints.

As part of the development process, running into errors can be time consuming. Especially when working with new areas such as frameworks and libraries. In this particular model, many of the machine learning libraries took time to understand and figure out how to accomplish specific tasks. Therefore, encountering error messages that require research and debugging.

On the cloud perspective, working with cloud services like AWS can also be a major challenge. The many services and use cases can be intimidating when choosing what resources benefit the architecture and how they actually work. Throughout this model, connecting the different layers was a challenge because AWS requires specific roles and permissions that lead to errors or no functionality when connecting between services. This includes establishing connections and security keys or certificates.

Creative problem solving is also required in any sort of research. For this particular model, packaging the trained machine learning models required us to research into docker containers, which allowed the model to work flawlessly in a packaged image in both local machine and AWS. This approach was taken due to the specific versions the local machine was running in comparison to some of the publicly available images that had different versions.

As part of the testing, not having the ability to test the model with actual fire data in real-time was also a challenge. To test performance metrics, the decision to use artificial data based off existing data had to be made to cover for limitations.

Finally, to conclude this section, the whole effort of research in existing literatures was also a major challenge. Each literature looks into different areas of research and determining what gaps exist can be a difficult thought process when integrating knowledge into new research that has not been explored.

5.2 Recommendations for Future Work

In extension of the work done in this thesis, there are potential directions that can be further explored. One of these is improving the model by adding notification features such as

SMS, VoIP, or email. By implementing different methods of notification, the model could improve communication between the users and alert properly when threat decisions are detected.

The implementation of this model did not focus intensively towards security, but integrating a login feature to the dashboard through the use of Cognito users could be explored. Other areas of security could also be explored to ensure protection of the model.

As discussed in previous sections, there are many sensors that could be used to detect fire threats. Adding other type of sensors to the model could be worth looking into. Especially the area of how techniques like fuzzy logic can be addressed to handle the addition of new sensors towards its logic, mainly due to the effect it would have on the fuzzy rules and memberships.

Optimizing the CNN model can be explored further. The current approach focuses on four categories within a multi-class classification model. Research can be done to split each category into its own CNN model and explore how lambda can direct to the specific model for more accurate results. This may remediate how performance metrics in the classes are affected by similar features.

As part of fault tolerance, implementing a decision-based backup could be implemented on the edge device in case AWS is not accessible at all. This could explore running the models on the edge device and store decisions until the cloud service is back up and running.

The GPS module is not used to full extent in this model, besides providing the coordinates of the threat location. Further can be explored to maximize use of the GPS data. For example, geofencing can be explored to help determine virtual boundaries in buildings that could help enhance the decision of indoor or outdoor threat. Also, the coordinates can be used in API based services like google earth or google maps to provide a visual aspect of the actual location.

On the network level, exploring how this model can be used as a node to create a large network of models spread throughout different locations can be researched further.

Finally, this model focused on a fuzzy logic and random forest classifier approach. Further research could be done to test how the model would perform in a random forest classifier singular design. The same could be explored by using a fuzzy logic independent approach to see how it would perform without RFC. By investigating these options further, a comparison can be done to determine if the fuzzy logic and random forest classifier combination is used to full extent as this was not covered in this work.

5.3 Summary

In this thesis, a hybrid fire detection system was developed by integrating Z-Score, fuzzy logic, random forest classifier, CNN, and an AWS infrastructure. The system leverages real-time sensor data and image analysis to identify fire threats throughout diverse conditions in outdoor and indoor domains. By collecting data through an edge device like the Raspberry Pi, the data is pre-processed and filtered by a Z-Score statistical approach. With a focus on anomalies, the data is stored in a DynamoDB database and processed through fuzzy logic and a random forest classifier endpoint to determine preliminary fire threats, which triggers the model to capture an image of environmental surroundings for further analysis. The CNN endpoint model introduced then evaluates incoming images to confirm fire related threats. After a decision is confirmed, the results are displayed on an interactive web application-based dashboard, which provides users with real-time updates that include sensor data and their predictions. The image is also displayed as part of the real-time status. This approach leverages AWS services to facilitate communication between the edge device and the cloud, while also ensuring scalability, and resource availability.

Through this implementation, a comprehensive solution bridges gaps in sensor and image-based fire detection by demonstrating effectiveness in fire threat identification. Additionally, this study also offers valuable insight into future improvements and potential research.

CONCLUSION

This thesis highlights the implementation of a hybrid fire detection model that integrates statistical analysis, fuzzy logic, machine learning models, and a robust AWS cloud-based infrastructure. By combining Z-Score normalization, fuzzy logic inference, a random forest classifier, and a CNN, the model effectively leverages real-time sensor data and image analysis to detect fire threats in diverse indoor and outdoor domains. The AWS integration enables seamless communication between an edge layer and the cloud, that provides scalability, reliability, and effective resource management. The proposed model demonstrates potential for hybrid approaches in fire detection where real time decision making is crucial. This work presents the practical approach of advanced technologies in addressing real world challenges. It also provides foundation for future research that may support diverse environmental conditions in fire detection by integrating sensor, image, and cloud-based solutions.

BIBLIOGRAPHY

- [1] V.S.Sanjana devi, S.k.B., Suresh kumar V, Vishwa K, Vaseegaran K, *Fire Detection and Prediction using Machine Learning for Fireworks Industry*, in *International Conference on Inventive Computation Technologies (ICICT)*. 2024, IEEE: Lalitpur, Nepal.
- [2] Xinyuan Wan, J.C., Biaohua Zhang, Xiaojian Xia, Jiceng Han, Kanghua Yan, *Machine Learning Method for Image Recognition-based Fire Detection System*, in *IEEE 6th Information Technology and Mechatronics Engineering Conference (ITOEC)*. 2022, IEEE: Chongqing, China.
- [3] Jaeseung Baek , T.J.A., Young-Seon Jeong, Myong K. Jeong , Senior Member, IEEE, Seongho Seo, Jongseok Kang, Yoseob Heo, *Intelligent Multi-Sensor Detection System for Monitoring Indoor Building Fires*. IEEE Sensors Journal, 2021. **21**(24): p. 27982 - 27992.
- [4] S. FOUZIYA SULTHANA , C.T.A.W., C. V. RAVIKUMAR, RAJESH ANBAZHAGAN, G. IDAYACHANDRAN , GIOVANNI PAU, *Review Study on Recent Developments in Fire Sensing Methods*. IEEE Access, 2023. **11**: p. 90269 - 90282.
- [5] Fathur Zaini Rachman, G.H., Wirawan, *A Fire Detection System Using Multi-Sensor Networks Based on Fuzzy Logic in Indoor Scenarios*, in *8th International Conference on Information and Communication Technology (ICoICT)*. 2020, IEEE: Yogyakarta, Indonesia.
- [6] Qian Su, G.H., Zhenxing Liu, *Research on fire detection method of complex space based on multi-sensor data fusion*. IOPScience, Measurement Science and Technology, 2024. **35**.
- [7] Seung Hwan Park, D.H.K., Sung Chul Kim, *Recognition of IoT-based fire-detection system fire-signal patterns applying fuzzy logic*. Heliyon, 2023. **9**(2).
- [8] Ms. G. Tejaswi, R.B., Sarvepalli Srihitha, Shaik Arshiha, Raja Venkata Sathya Sarayu, *PREDICTING FIRE ALARMS USING MULTI SENSOR DATA: A BINARY CLASSIFICATION APPROACH*. Turkish Journal of Computer and Mathematics Education, 2024. **15**(1): p. 242-255.
- [9] Giacomo Peruzzi, A.P., Mattia Van Der Meer, *Fight Fire with Fire: Detecting Forest Fires with Embedded Machine Learning Models Dealing with Audio and Images on Low Power IoT Devices*. Sensors, 2023. **23**(2).
- [10] Yuen Chark See, E.X.H., *IoT-Based Fire Safety System Using MQTT Communication Protocol*. INTERNATIONAL JOURNAL OF INTEGRATED ENGINEERING, 2020. **12**(6): p. 207-215.

- [11] Wen Jiang, C.X., Miaoyan Zhuang, Yehang Shou, Yongchuan Tang, *Sensor Data Fusion with Z-Numbers and Its Application in Fault Diagnosis*. Sensors, 2016. **16**(9).
- [12] Shome, S.J.a.S.K., *Hybrid Ensemble Based Machine Learning for Smart Building Fire Detection Using Multi Modal Sensor Data*. Fire Technology, 2022. **59**: p. 473-496.
- [13] Xinyuan Wan, J.C., Shengxiang Luo, Zhengxing Tian, Li Zhang, Xiaojian Xia, *Gaussian Process for the Machine Learning-based Smart fire Detection System*, in *IEEE 6th Information Technology and Mechatronics Engineering Conference (ITOEC)*. 2022, IEEE: Chongqing, China.
- [14] Saptajit Banerjee, R.D., Rajesh Rathinam, R. Dhanalakshmi, *Real-Time Fire Detection in Unmanned Ground Vehicles Integrating YoloV5 and AWS IoT*, in *International Conference on System, Computation, Automation and Networking (ICSCAN)*. 2023, IEEE: PUDUCHERRY, India.
- [15] Qingjuan Wang, J.L., Yan Zhou, Huibin Luo, Ren Zhao, *Indoor Fire Detection System Based on Multi-sensor Information Fusion Algorithm*, in *IEEE 4th International Conference on Pattern Recognition and Machine Learning (PRML)*. 2023, IEEE: Urumqi, China.
- [16] Priyanka Kushwaha, M.S., Pragati Kumari, Richa Yadav *Multi-sensor Data Fusion for Early Fire Estimation Using ML Techniques*, in *International Conference on Power Engineering and Intelligent Systems (PEIS)*. 2023, Springer, Singapore. p. 267-277.
- [17] Diaconu, B.M., *Recent Advances and Emerging Directions in Fire Detection Systems Based on Machine Learning Algorithms*. Fire, 2023. **6**(11)
- [18] Amazon Web Services (AWS) – Cloud Computing Services. <https://aws.amazon.com/>
- [19] Abdur Rouf, E.A., Batyr Charyyev, *Latency and Bandwidth Benefits of Edge Computing for Scientific Applications*, in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. 2024, IEEE: Vancouver, BC, Canada.
- [20] Neha Kewate, A.R., Mohit Dubekar, Yuvraj Raut, Prof. Ankush Patil, *A Review on AWS - Cloud Computing Technology*. International Journal for Research in Applied Science & Engineering Technology (IJRASET), 2022. **10**(1).
- [21] Izeboudjen, F.A.a.N., *Predicting Forest Fire in Algeria Using Data Mining Techniques: Case Study of the Decision Tree Algorithm*, in *International Conference on Advanced Intelligent Systems for Sustainable Development*. 2020, Springer, Cham. p. 363-370.
- [22] Kaggle, *Fire Detection Dataset*. <https://www.kaggle.com/datasets/atulyakumar98/test-dataset>

- [23] Kaggle, *Fire Images Dataset (Indoor and Outdoor)*.
<https://www.kaggle.com/datasets/rachadlakis/firedataset-jpg-224?resource=download>
- [24] Kaggle, *FIRE Dataset*. <https://www.kaggle.com/datasets/phylake1337/fire-dataset>
- [25] Kaggle, *Fire and Smoke Dataset*. <https://www.kaggle.com/datasets/dataclusterlabs/fire-and-smoke-dataset>
- [26] Kaggle, *Indoor Images*. <https://www.kaggle.com/datasets/sshubhamsingh/indoor-images>
- [27] Kaggle, *Day and Night Image*. <https://www.kaggle.com/datasets/ibrahimalobaid/day-and-night-image>
- [28] Kaggle, *Time of Day Dataset*.
<https://www.kaggle.com/datasets/aymenkhoulja/timeofdaydataset?select=nighttime>
- [29] Kaggle, *MIT Indoor Scenes*. <https://www.kaggle.com/datasets/itsahmad/indoor-scenes-cvpr-2019/data>
- [30] Kaggle, *Google Street View*.
<https://www.kaggle.com/datasets/paulchambaz/google-street-view>
- [31] Science Data Bank, *An open flame and smoke detection dataset for deep learning in remote sensing based fire detection*.
<https://www.scidb.cn/en/detail?dataSetId=ce9c9400b44148e1b0a749f5c3eb0bda>
- [32] ResNet50 TensorFlow,
https://www.tensorflow.org/api_docs/python/tf/keras/applications/ResNet50

VITA

Jose A. Burnias is originally from San Juan, Tx. In December 2016 he earned his bachelor's degree in computer engineering from the University of Texas at Rio Grande Valley. After completing his engineering degree, he moved on to work as a Software Engineer for Accenture Federal Services, which he has been a part of for the past 7 years. In Fall 2020 he started pursuing his master's degree in Computer Engineering at the University of Texas at San Antonio. His interest in cloud computing led him to work under the supervision of Dr. Jeff Prevost as part of his graduate level studies. He plans to continue growing in the Software Engineering and cloud computing fields to become proficient in developing architectural solutions for the industry. His plans also consist of eventually pursuing a doctoral degree in the future.